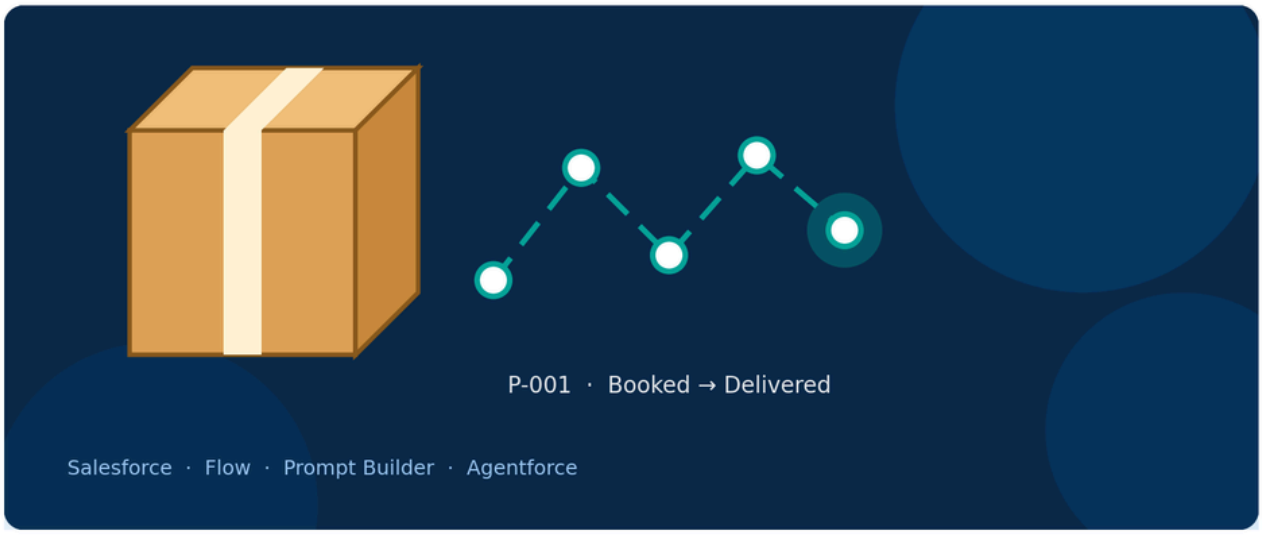
SwiftShip Tracker



Platform Environment	Salesforce Developer Edition
Core Custom Modules	Parcel (Parcel__c), Delivery (Delivery__c), Sender (Sender__c), Receiver (Receiver__c)
Automation Engine	Salesforce Flow Builder (Auto-Launched Flows)
Artificial Intelligence Layer	Agentforce + Salesforce Prompt Builder
User Interface Architecture	Salesforce Lightning Application & Dedicated Lightning Record Pages
Security Architecture	Profiles, Permission Sets, Role Hierarchy, Field-Level Security (FLS)
Project Type & Context	Academic / Prototype Implementation
Academic Department	Department of Computer Science and Engineering
Implementation Timeline	Academic Year 2026–2027

Contents

1. ProjectBackground and Operational Context
2. Requirements Analysis and System Objectives
3. Enterprise Architectural Framework
4. Data Architecture and Salesforce Custom Schema
5. Conversational AI Integration: Prompt Builder Mechanics
6. Process Automation Architecture: Flow Builder Orchestration
7. Conversational Agent Configuration: Agentforce Deployment
8. Security Architecture and Governance Controls
9. Quality Assurance, Test Execution, and Empirical Validation
10. System Evaluation: Advantages and Constraints
11. Strategic Roadmap and Future Logistics Integration
12. Requirements Traceability Matrix
13. User Guide
14. Administrator Deployment and Maintenance Guide
15. Risk Register and Troubleshooting
16. Conclusion

Appendix A. Implementation Execution Checklist

Appendix B. Representative Conversational AI Output Payload

Appendix C. Source Code Listings (C.1 to C.12)

Appendix D. Glossary

Appendix E. References and Document History

1. Project Background and Operational Context

Inmodern logistics and enterprisesupply chainmanagement, centralising fragmented parcel tracking mechanisms into a unified Customer Relationship Management (CRM) architecture is critical for operational efficiency, data integrity, and customer retention. Logistics enterprises frequently operate across disparate operational silos, where booking details, transit milestones, sender/receiver profiles, and delivery status updates reside in disconnected software applications or unindexed spreadsheets. Unifying these mechanisms within an enterprise CRM platform establishes a definitive single source of truth. This centralised architecture provides real-time operational visibility across the complete parcel lifecycle, eliminates manual administrative overhead, accelerates support desk response times, and builds a foundation for scalable workflow automation and generative artificial intelligence.

The traditional operational baseline in legacy logistics environments is severely constrained by information fragmentation. When parcel tracking data is distributed across manual logbooks, isolated spreadsheets, telephone support queues, and proprietary localised software, severe operational friction ensues. Field delivery personnel lack real-time mechanisms to log status transitions, while customer support teams must execute manual lookups across disconnected databases merely to address basic status inquiries. This results in high administrative costs, elevated data-entry error rates, delayed issue resolution, and degraded customer trust.

To overcome these challenges, the SwiftShip Tracker platform was architected as an AI-enabled parcel management prototype on the Salesforce platform. The primary strategic objective is to resolve tracking fragmentation by integrating parcel booking, status management, stakeholder recording, and conversational customer support into a unified Salesforce architecture.

Strategic Objectives

- Centralise parcel, delivery, sender, and receiver records within a relational CRM data schema.
- Streamline end-to-end parcel lifecycle tracking from booking to final delivery confirmation.
- Provide real-time shipment status and estimated delivery metrics to internal staff and customer service channels.
- Automate operational data retrieval and payload routing through low-code process flows.
- Enable natural language conversational tracking using grounded generative AI models.
- Establish data foundations to drive executive visibility through native Salesforce reports and dashboards.
- Enforce enterprise-grade security controls and field-level access boundaries across all user personas.

Implementation Scope

- Data modelling and custom object configuration for Parcel__c, Delivery__c, Sender__c, and Receiver__c.
- Design, grounding, and activation of AI prompt templates in Prompt Builder.
- Configuration of auto-launched orchestration flows in Flow Builder to connect CRM data records to AI actions.
- Implementation of governance controls, including custom permission sets for Flow access execution.
- Development of custom Lightning tabs and record page layouts for streamlined user navigation.
- Rigorous empirical validation and test execution within a controlled Salesforce Developer Edition environment.

2. Requirements Analysis and System Objectives

Rigorous requirement definition is fundamental to platform architecture. Formalising functional specifications and quality attributes prevents architectural drift, aligns engineering tasks with operational goals, and ensures that automated workflows directly support business processes. The functional capabilities were synthesised into twelve requirement specifications (FR-01 to FR-12).

ID	Requirement Domain	Operational Description and Impact
FR-01	Record Management	System must support full lifecycle management (creation, update, retrieval) of custom parcel records within Salesforce.
FR-02	Data Identification	System must generate and enforce unique Parcel ID values across all records to guarantee exact indexing and lookup.
FR-03	Lifecycle Tracking	System must maintain core parcel attributes, including status picklist values, package weight, and estimated delivery dates.
FR-04	Stakeholder Management	System must maintain dedicated sender and receiver records linked directly to corresponding parcel records.
FR-05	Delivery Operations	System must support delivery operational tracking, including real-time location metrics and delivery estimate adjustments.
FR-06	Data Retrieval	System must retrieve specific parcel records based on an incoming unique Parcel ID text parameter.

ID	Requirement Domain	Operational Description and Impact
FR-07	Conversational Output	System must generate formatted, natural language conversational tracking updates for customer inquiries.
FR-08	Automation Integration	System must utilise Salesforce Flow Builder to orchestrate data movement between parcel records and Prompt Builder actions.
FR-09	Access Governance	System must enforce granular user access controls to objects, fields, and flows via custom Permission Sets.
FR-10	Interface Navigation	System must provide custom Lightning tabs and structured record layouts to streamline operational navigation.
FR-11	AI Agent Deployment	System must support Agentforce configuration to expose parcel tracking capabilities through conversational AI interfaces.
FR-12	Operational Analytics	System must construct structured custom object data models that serve as the foundation for native Salesforce reports and dashboards.

Table 1. Functional requirements FR-01 to FR-12

Non-Functional Quality Attributes

- **Usability:** Standard Salesforce LightningDesign System (SLDS) components deliver clear, consistent interfaces for internal teams and intuitive, natural language responses for conversational AI.
- **Security:** Defence-in-depth combining profile constraints, custom permission sets, role hierarchies, and Field-Level Security (FLS).
- **Reliability:** Process automation and prompt generation must execute deterministically, ensuring accurate data retrieval and zero AI hallucination within platform governor limits.
- **Performance:** Flow execution and Prompt Builder dynamic generation must complete within strict sub-second latency thresholds to support real-time agent interactions.
- **Scalability:** The custom object model and low-code flows must accommodate growing transactional volumes, user counts, and high-frequency queries.
- **Maintainability:** Custom objects, prompt templates, auto-launched flows, and permission sets are built modularly to simplify future updates.
- **Data Integrity:** Relational constraints, lookup fields, auto-numbering, and enforced picklist value sets preserve consistency across all lifecycle stages.

3. Enterprise Architectural Framework

The SwiftShip Tracker is structured as a multi-tiered enterprise framework. Decoupling presentation, application logic, data storage, process orchestration, generative AI processing, security governance, and analytics ensures that individual tiers, such as prompt templates or background flows, can be optimised or expanded without destabilising the core schema or its data access security. The eight structural layers are shown below.

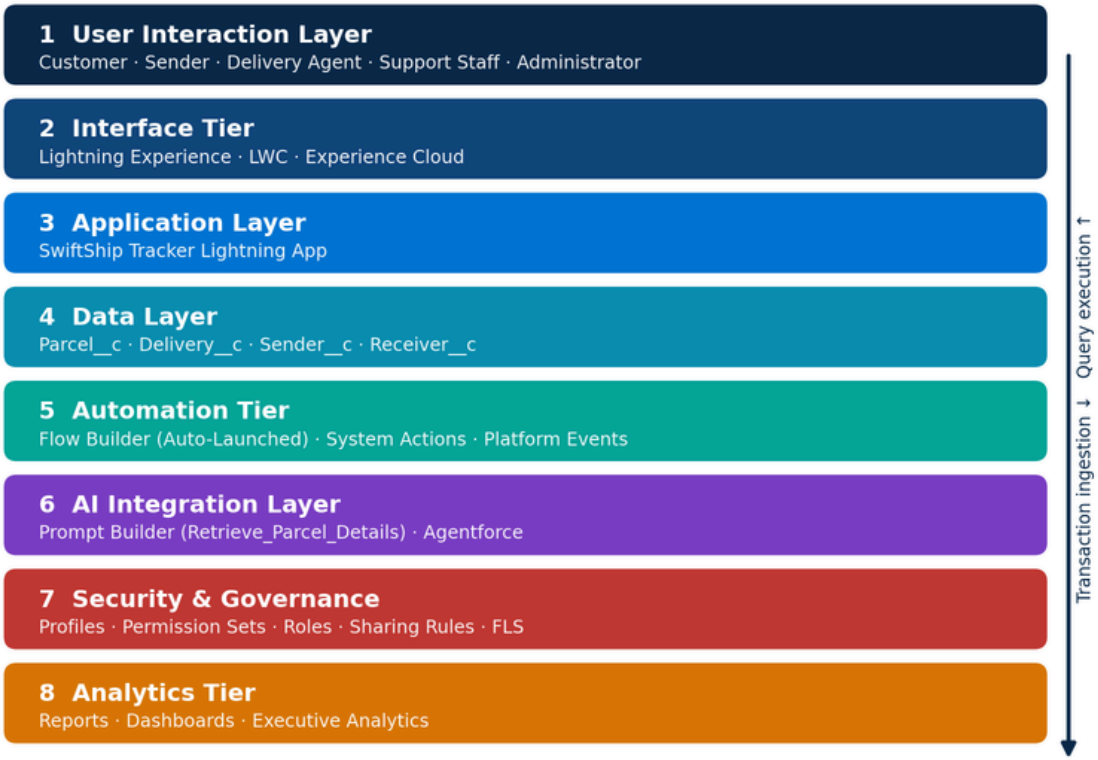


Figure 1. Eight-layer enterprise architecture of the SwiftShip Tracker platform

Data flows downward through this hierarchy during transaction ingestion and upward during query execution. For example, a conversational query submitted at the Interface Tier passes to the AI Integration Layer, triggers the Automation Tier to query the Data Layer under strict Security & Governance enforcement, and returns a grounded response to the user.

Parcel Lifecycle

Every parcel record transitions through a deterministic sequence of operational states, from booking to post-delivery support.

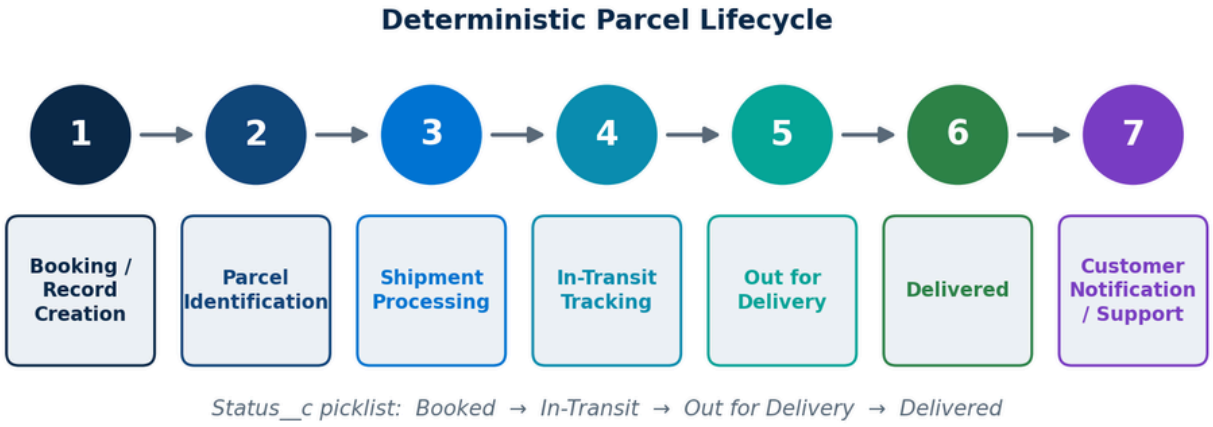


Figure 2. Deterministic parcel lifecycle and the Status__c picklist

#	Stage	Description
1	Parcel Booking / Record Creation	Initial entry of parcel dimensions, weight, sender, and receiver details into the CRM database.

#	Stage	Description
2	Parcel Identification	System generation and assignment of the unique indexing identifier (Parcel_ID__c) used for tracking lookups.
3	Shipment Processing	Facility sorting, package aggregation, and initial dispatch preparation.
4	In-Transit Tracking	Active transport phase, capturing dynamic location transitions and route updates on linked Delivery__c records.
5	Out for Delivery	Final-leg assignment where the parcel is dispatched with field couriers.
6	Delivered	Verification and system confirmation of parcel receipt, terminating active logistics tracking logic.
7	Customer Notification / Support	Post-delivery phase enabling automated confirmation notifications and AI-assisted conversational support.

Table 2. Parcel lifecycle stages

4. Data Architecture and Salesforce Custom Schema

Thedata architecture is implemented with customSalesforce objectsconfigured in ObjectManager. Normalised relational design preserves data integrity, models enterprise business relationships, and exposes reliable record context to AI prompt grounding. Isolating logistics entities into discrete objects eliminates redundancy and enforces strict normalisation.

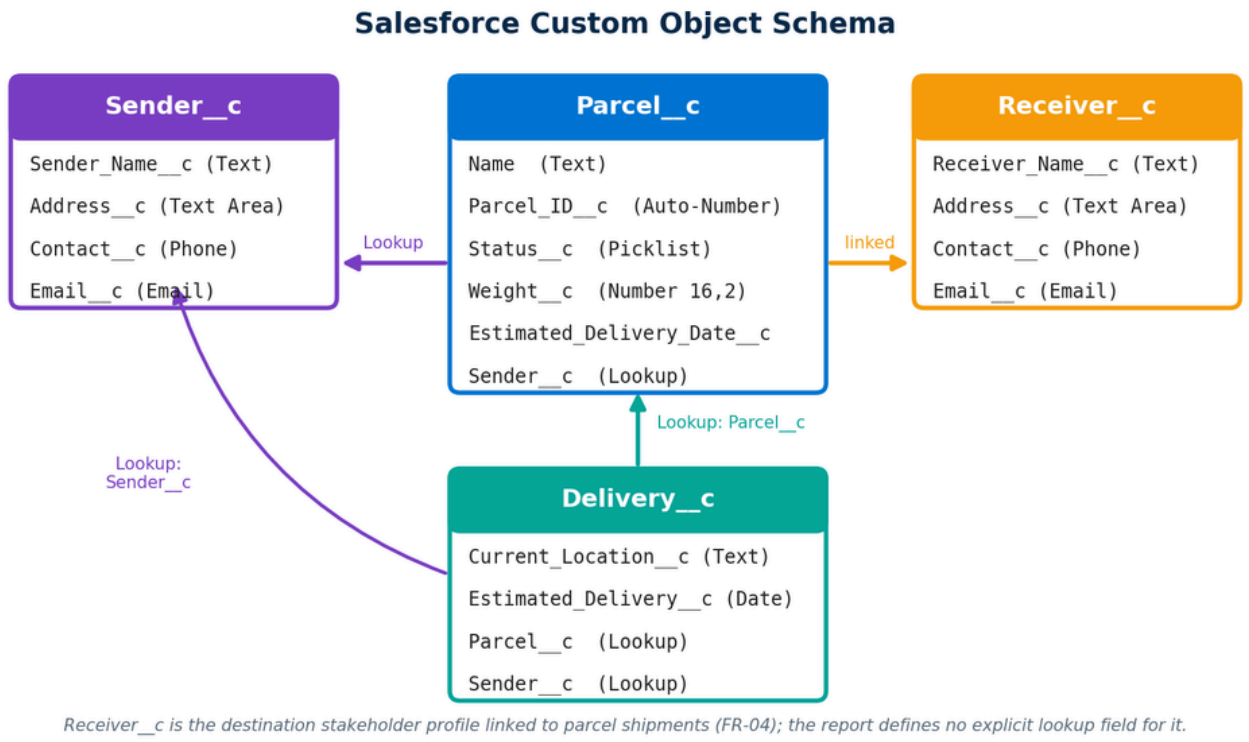


Figure 3. Relational schema of the four core custom objects

Object	API Name	Role	Description and Core Fields
Parcel	Parcel__c	Primary Operational Entity	Core shipment record holding tracking parameters, physical attributes, status, and lookup links. Fields: Name (Text), Parcel_ID__c (Auto-Number), Status__c (Picklist), Weight__c (Number), Estimated_Delivery_Date__c (Date), Sender__c (Lookup).

Object	API Name	Role	Description and Core Fields
Delivery	Delivery__c	Logistics Tracking Entity	Stores active delivery tracking data and location milestones. Fields: Current_Location__c (Text), Estimated_Delivery__c (Date), Parcel__c (Lookup to Parcel__c), Sender__c (Lookup to Sender__c).
Sender	Sender__c	Stakeholder Origin Entity	Origin party contact attributes and address details. Fields: Sender_Name__c (Text), Address__c (Text/TextArea), Contact__c (Phone), Email__c (Email).
Receiver	Receiver__c	Stakeholder Destination Entity	Destination party attributes, contact information, and final delivery locations. Fields: Receiver_Name__c (Text), Address__c (Text/TextArea), Contact__c (Phone), Email__c (Email).

Table 3. Core custom objects

Parcel__c Field Specification

Field Label	API Name / Type	Constraint	Purpose
Parcel Name	Name (Text)	Standard field, Text(80)	Display label for the package record (e.g., Laptop, Sandwood).
Parcel ID	Parcel_ID__c (Auto-Number)	Auto-number, display format P-{000}	System-generated tracking ID (P-001, P-002…) ensuring indexed key retrieval.
Status	Status__c (Picklist)	Booked, In-Transit, Out for Delivery, Delivered	Current operational lifecycle state for routing, filtering, and customer support.
Weight	Weight__c (Number)	Number(16, 2)	Physical package mass in kilograms for routing and capacity planning.
Estimated Delivery	Estimated_Delivery_Date__c (Date)	Standard Date field	Target delivery date for milestone management and AI grounding.
Sender	Sender__c (Lookup)	Lookup(Sender__c)	Connects the package to the origin stakeholder profile.

Table 4. Parcel__c field-level specification

To expose these objects to operational staff, logistics managers, and customer service teams, dedicated Custom Object Tabs were created for Parcel__c, Sender__c, and Receiver__c and configured within the custom “SwiftShip Tracker” Lightning Application. Users can navigate across shipment profiles, update status picklists, manage stakeholder accounts, and view operational records without complex navigation.

Data Dictionary

The tablesbelowconsolidate every field of the four custom objects as built in Object Manager, together with the object-level settings visible in the Developer Edition org. Field API names follow the Salesforce __c convention.

Field label	API name	Type	Notes
Parcel Name	Name	Text(80)	Standard name field; human readable label such as Laptop.
Parcel ID	Parcel_ID__c	Auto-Number	Display format P-{000}; observed values P-001, P-002, P-005.
Status	Status__c	Picklist	Booked, In-Transit, Out for Delivery, Delivered.

Field label	API name	Type	Notes
Weight	Weight__c	Number(16,2)	Kilograms.
Estimated Delivery	Estimated_Delivery_Date__c	Date	Used for milestone management and AI grounding.
Sender	Sender__c	Lookup(Sender)	Origin stakeholder.

Table 5. Parcel__c data dictionary

Field label	API name	Type	Notes
Current Location	Current_Location__c	Text	Latest location milestone.
Estimated Delivery	Estimated_Delivery__c	Date	Delivery-level estimate; may differ from the parcel estimate.
Parcel	Parcel__c	Lookup(Parcel)	Links the milestone to its shipment.
Sender	Sender__c	Lookup(Sender)	Origin stakeholder.

Table 6. Delivery__c data dictionary

Field label	API name	Type	Notes
Sender / Receiver Name	Sender_Name__c / Receiver_Name__c	Text	Party name.
Address	Address__c	Text / Text Area	Origin or destination address.
Contact	Contact__c	Phone	Sensitive: restricted by FLS.
Email	Email__c	Email	Sensitive: restricted by FLS.

Table 7. Sender__c and Receiver__c data dictionary (identical structure)

Setting API name	Delivery	Sender
Singular / plural label	Delivery__c	sender__c
Custom object	Delivery / Deliveries	sender / senders
Enable Reports	Yes	Yes
Track Activities	Enabled	Enabled
Track Field History	Enabled	Enabled
Deployment status	Enabled	Enabled
	Deployed	Deployed

Table 8. Object settings observed in Object Manager

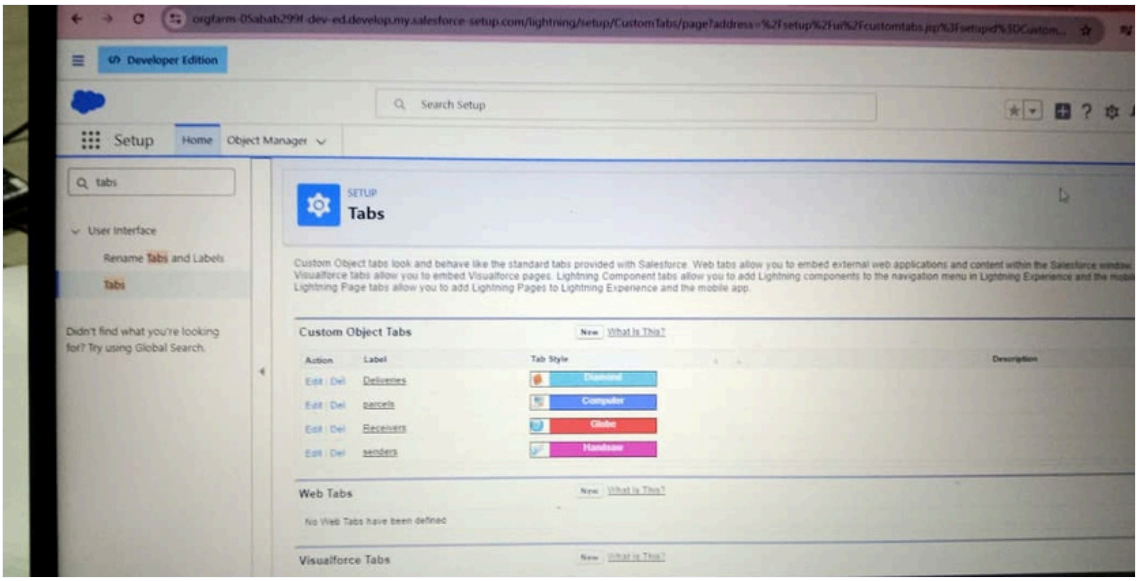


Figure 4. Custom Object Tabs in Setup: Deliveries, parcels, Receivers and senders, each with its own tab style.

The Tabs page lists four custom object tabs. The report describes tabs for Parcel, Sender and Receiver; a fourth tab for Deliveries was also created, giving field staff direct navigation to delivery milestones. Because the objects were created with lower-case labels, the tabs display as “parcels” and “senders”; renaming the labels would make the interface more consistent.

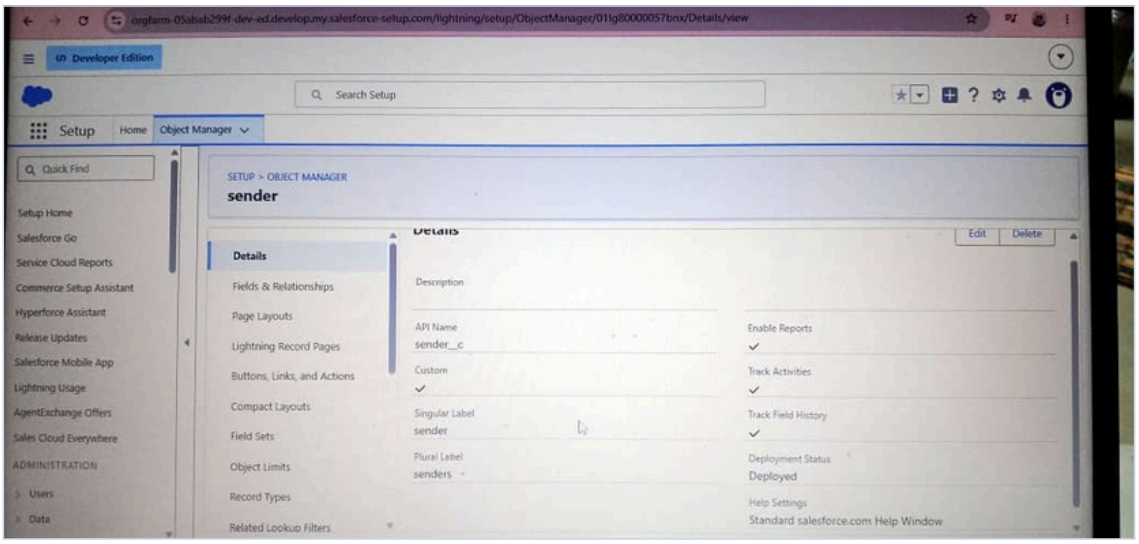


Figure 5. Object Manager details for the Sender object (API name sender__c, Deployed).

The Sender object has reports, activity tracking and field history tracking enabled. Field history tracking is valuable for audit purposes, for example to see who changed a sender address.

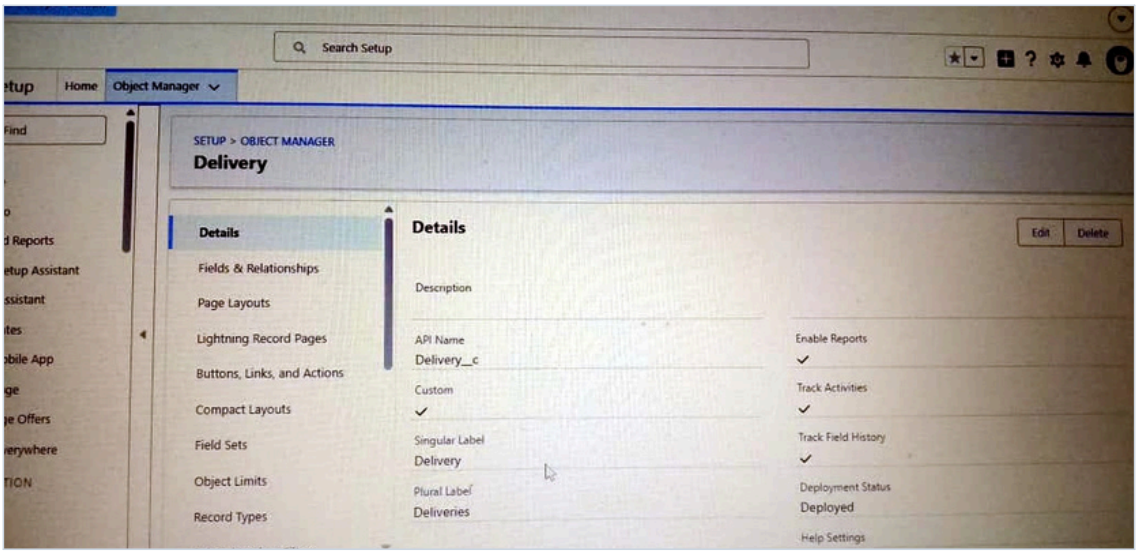


Figure 6. Object Manager details for the Delivery object (API name Delivery__c, Deployed).

Prototype Test Records

Parcel ID	Name	Status	Weight	Estimated delivery	Source
P-001	Laptop	Booked	2	09/17/2026	Record page and Prompt Builder preview
P-002	Sandwood	Out for Delivery	5	09/26/2026	Record page
P-005	P-005 (as resolved in preview)	Booked	2	10/17/2026	Prompt Builder preview

Table 9. Prototype records observed in the org

5. Conversational AI Integration: Prompt Builder Mechanics

Deploying generative AI in enterprise logistics requires strict dynamic grounding. Standard large language models operating without context can hallucinate shipment states, misinforming customers. SwiftShip Tracker addresses this by using Salesforce Prompt Builder to bind prompt templates directly to live CRM record fields, so AI responses reflect database truth.

Configuration Parameter	Specification
Template Name	Retrieve Parcel Details
API Name	Retrieve_Parcel_Details
Functional Purpose	Format underlying Parcel__c database fields into natural language tracking summaries.
Input Context	Parcel__c record context / passed Parcel_ID__c filter parameter
Output Format	Formatted natural language parcel tracking update

Table 10. “Retrieve Parcel Details” prompt template configuration

The dynamic grounding expression configured in the template is:

```
Parcel Tracking Update
• Parcel Name: {!$Input:Parcel__c.Name}
• Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
```

- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c}
- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}

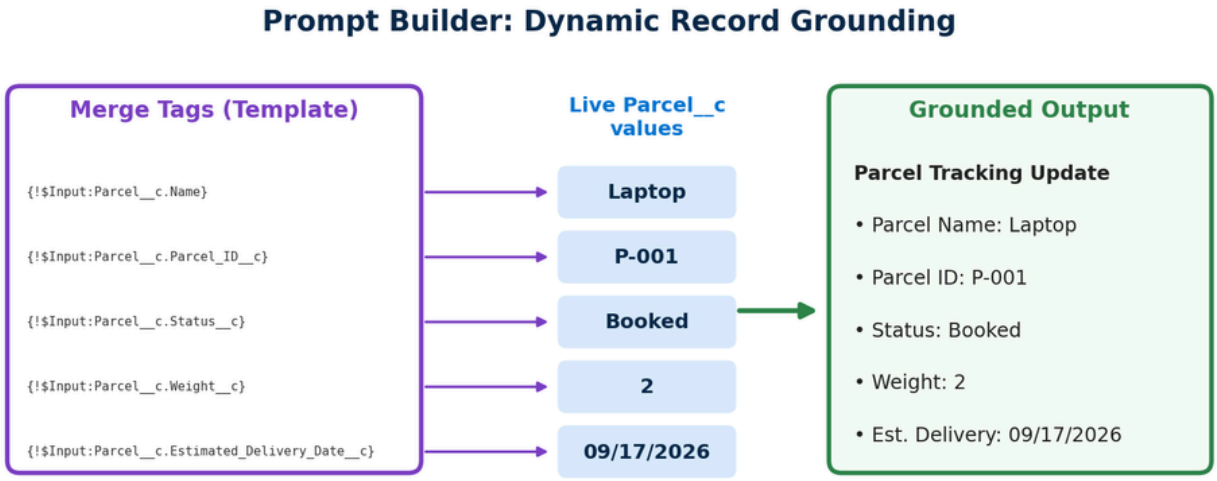


Figure 7. Merge tags resolve against live Parcel__c values to produce a grounded response

When invoked, the Prompt Builder engine ingests the Parcel__c record ID, substitutes field values into the merge tags, and returns a fully grounded textual tracking summary. Because merge resolution binds directly to database fields, the output is tied to stored record values rather than model-generated content.

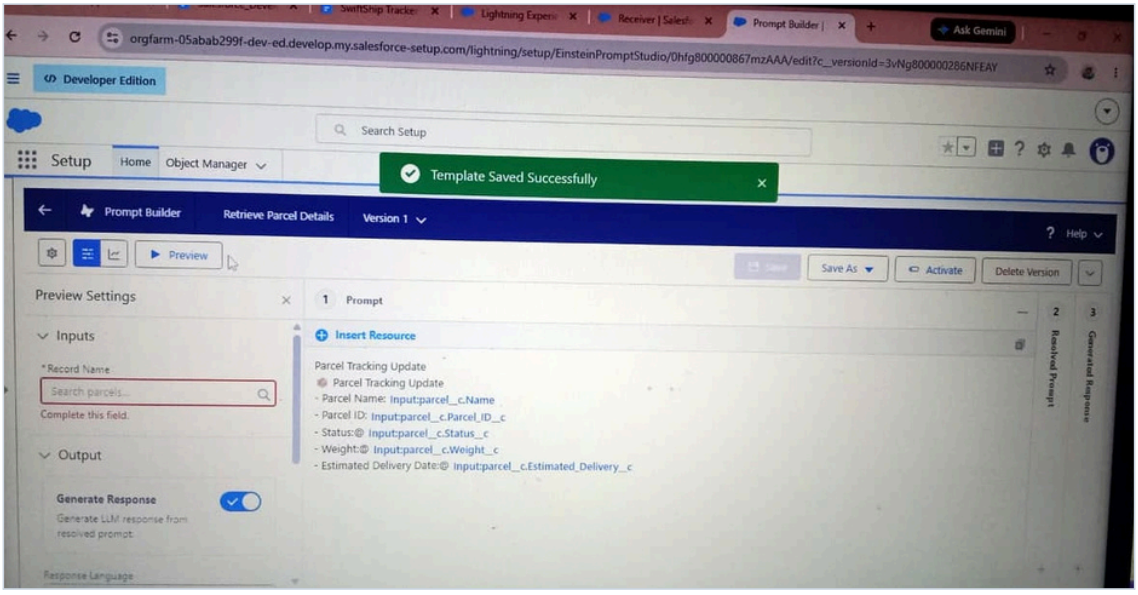


Figure 8. Retrieve Parcel Details prompt template in Prompt Builder, saved successfully (Version 1).

The template body uses five merge fields from the Parcel__c record. The Preview Settings panel requires a Record Name, which is the Parcel__c record used to resolve the template. The Generate Response toggle is on, meaning the resolved prompt is passed to the large language model (LLM) to write the final answer.

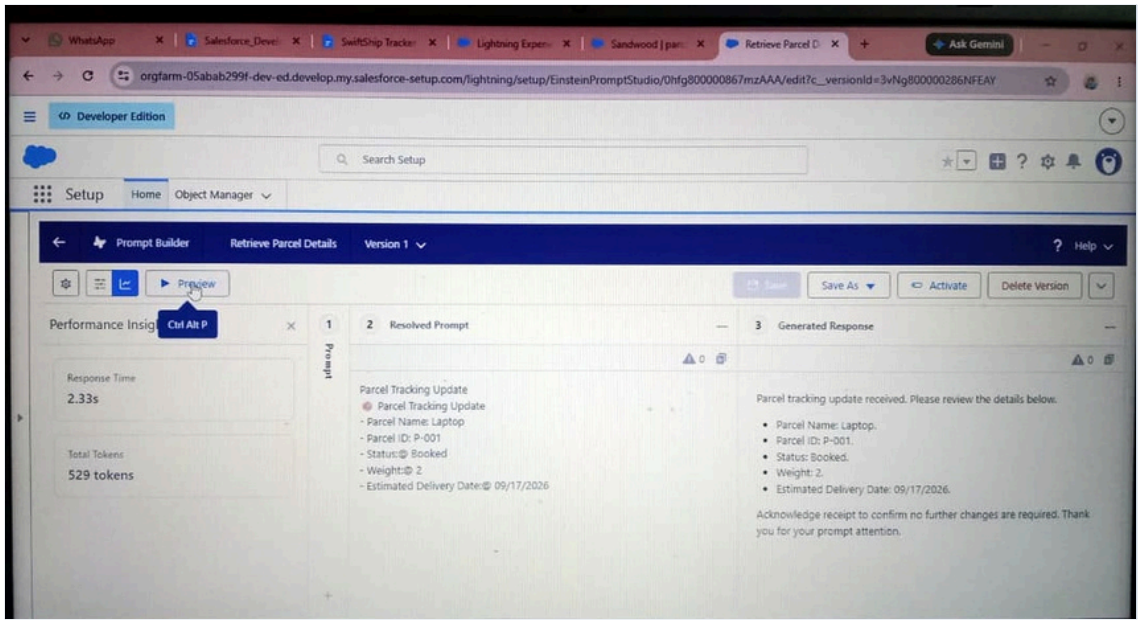


Figure 9. Preview using the Laptop record: Resolved Prompt (left) and Generated Response (right).

The Resolved Prompt panel shows the merge fields replaced by stored values: Laptop, P-001, Booked, 2 and 09/17/2026. The Generated Response repeats these values exactly. The performance panel reports a response time of 2.33 seconds and 529 tokens.

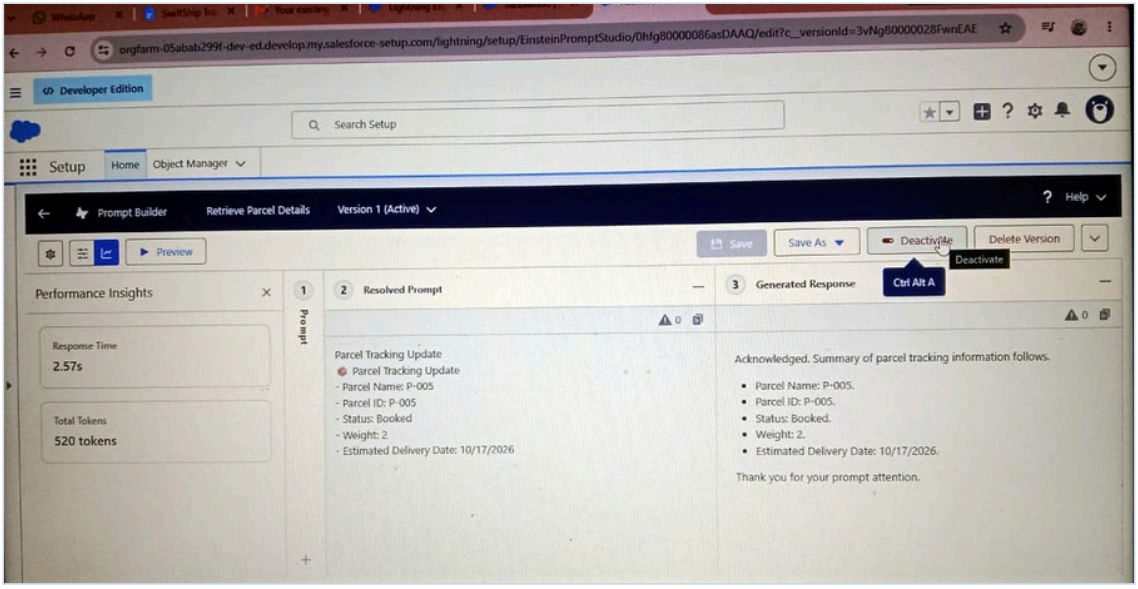


Figure 10. Template Active (Version 1) and previewed with record P-005.

After activation the template shows the status Active and a Deactivate button. The preview for P-005 again reproduces all five field values (Booked, weight 2, 10/17/2026) with a response time of 2.57 seconds and 520 tokens.

Observations from Prompt Testing

Observation	Evidence	Implication
Field values are reproduced exactly	Resolved and generated panels match for P-001 and P-005.	Merge-field grounding works; data comes from the record, not from model memory.
LLM adds filler sentences	Responses include phrases such as acknowledging receipt and thanking the reader.	Tighten the prompt to forbid extra text (see Appendix C.5).

Observation	Evidence	Implication
Latency of 2.3 to 2.6 seconds	Performance Insights panel.	Exceeds the sub-second target in the non-functional requirements; see below.
Token use of about 520 to 530	Performance Insights panel.	Small prompts keep cost and latency low; avoid unnecessary context.

Table 11. Prompt Builder observations

Two points deserve to be stated plainly. First, the guarantee of “zero hallucination” applies to the field values, which are supplied by the record; it does not extend to the surrounding wording, because an LLM writes that. Second, the measured latency of roughly 2.5 seconds is the cost of the LLM step. The non-functional requirement should therefore be read as “sub-second for data retrieval and a few seconds for generated prose”. A deterministic Apex action (Section 7) retrieves the same fields without an LLM in the data path.

6. Process Automation Architecture: Flow Builder Orchestration

Salesforce Flow Builder is the orchestration middle layer. While Prompt Builder defines the format of AI output, an automated process must receive incoming queries, execute database lookups, pass record context to the prompt template, and route the generated response back to the caller. The auto-launched flow “Parcel Details Flow” (Parcel Details – V1) performs this role.

Resource	Type	Functional Role
ids	Text Variable (Input)	Receives the target Parcel_ID__c string passed from external callers, scripts, or Agentforce actions.
Output	Text Variable (Output)	Captures and exposes the final natural language tracking response returned by Prompt Builder.

Table 12. Flow interface variables

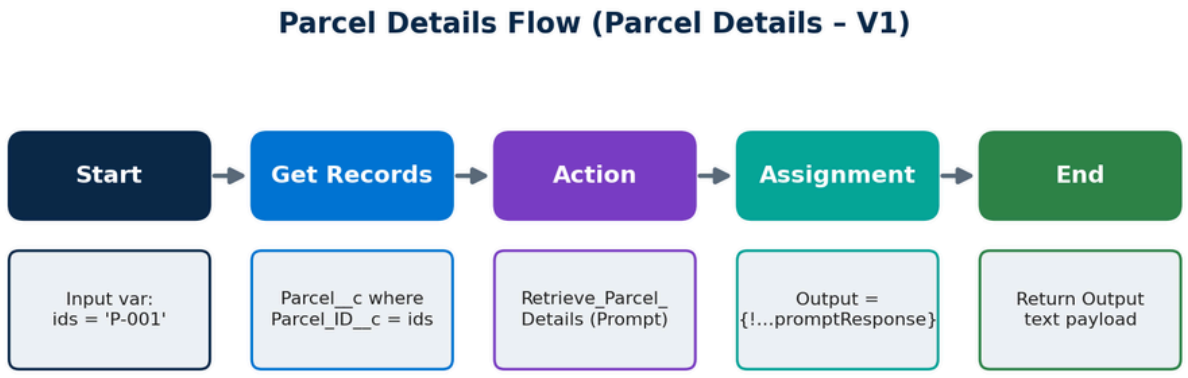


Figure 11. Single-path node-based execution of the Parcel Details Flow

- **Flow Invocation:** an external agent or process passes an input string (e.g., 'P-001') into the ids variable.
- **Data Ingestion (Get Records):** the flow queries Parcel__c for a record where Parcel_ID__c equals the incoming ids value.
- **AI Action Invocation (Action):** the flow calls the Prompt Builder action Retrieve_Parcel_Details, passing the retrieved Parcel__c record as input context.

- **Variable Assignment (Assignment):** the returned text is assigned to the output variable (Output = {!Retrieve_Parcel_Details.promptResponse}).
- **Termination and Payload Return:** the populated Output text is returned to the calling agent or user interface.

The Assignment element acts as an integration gateway, mapping internal AI prompt results into standard, consumable flow variables. This decouples AI generation logic from front-end channels, allowing any calling system to request tracking updates cleanly.

Flow Element Specification

#	Element	Type	Configuration
1	Autolaunched Flow	Start	No trigger; invoked by an agent action or another process.
2	Get Parcel Records	Get Records	Object Parcel__c, filter Parcel_ID__c equals {!ids}; stores the matching record.
3	Parcel Updates (Parcel_Updates)	Action	Prompt template Retrieve Parcel Details; objectToSummarize = parcel from Get Parcel Records.
4	Assignment Output (Assignment_Output)	Assignment	Output equals Parcel Updates > Prompt Response.
5	End	End	Returns the Output text variable.

Table 13. Parcel Details – V1 element specification

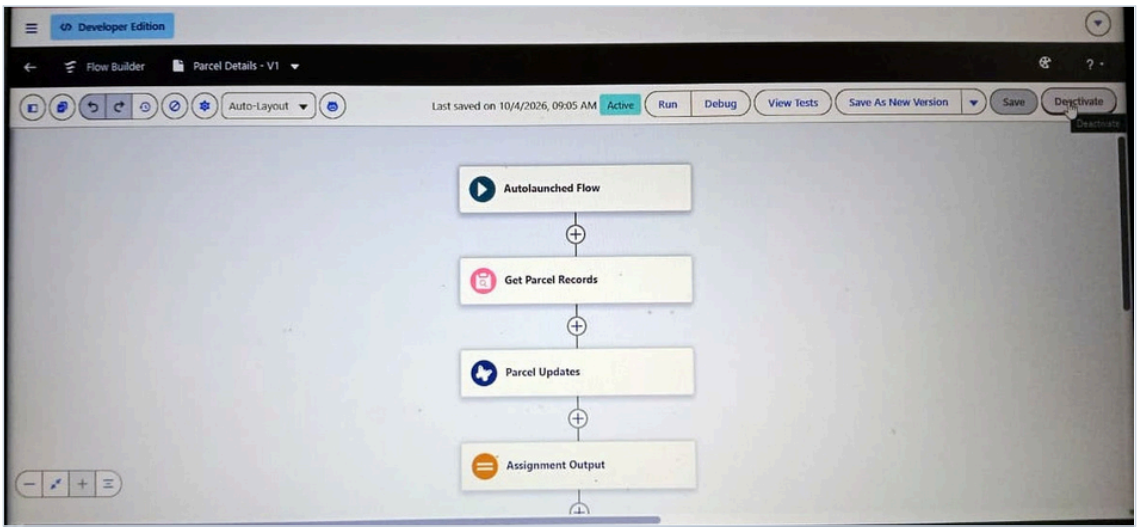


Figure 12. Parcel Details – V1 in Flow Builder, status Active (last saved 10/4/2026, 09:05 AM).

The canvas shows the single-path structure: Autolaunched Flow, Get Parcel Records, Parcel Updates and Assignment Output. The Active badge confirms that the flow can be invoked by agents and other callers.

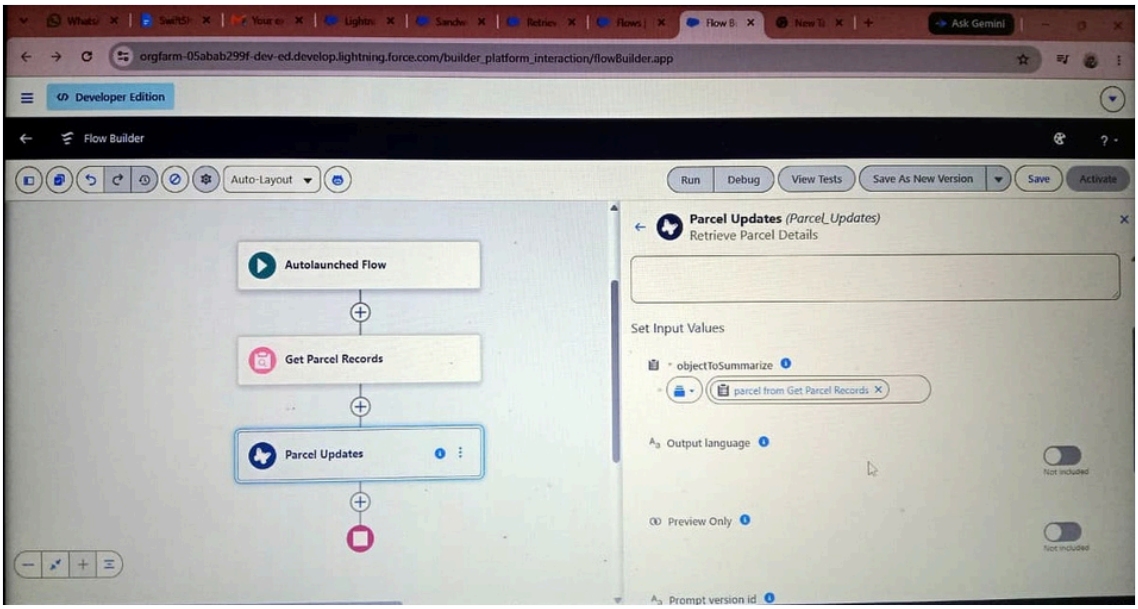


Figure 13. Parcel Updates action: the prompt template input objectToSummarize is the record from Get Parcel Records.

The action element exposes the optional inputs Output language and Preview Only; both are left as Not included. Passing the Get Records result as objectToSummarize is what grounds the prompt in the stored record.

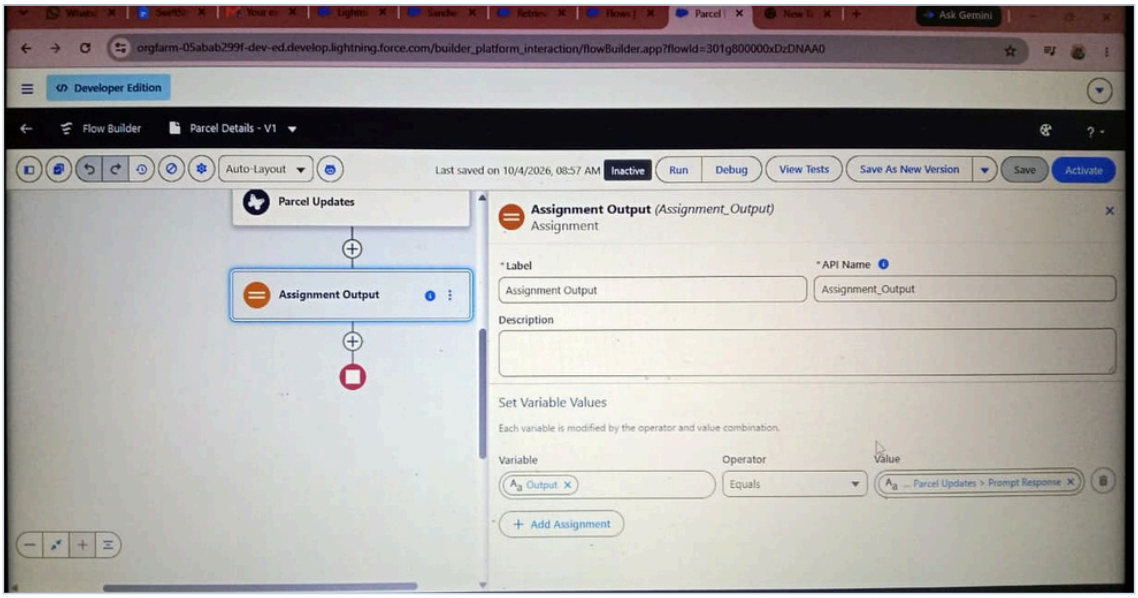


Figure 14. Assignment Output element: Output equals the Parcel Updates prompt response (saved, status Inactive at 08:57).

This capture was taken before activation; the previous figure shows the same flow active at 09:05. The assignment copies the generated text into the Output variable so that any caller can read it.

Handling a Parcel That Does Not Exist

The flow as built has no branch for an empty Get Records result. If the supplied ID matches nothing, the prompt action receives no record. TC-08 requires a graceful no-match response. The recommended change is a Decision element after Get Parcel Records: when the record is found, continue to Parcel Updates; otherwise assign Output a fixed message such as “No parcel found with that ID.” The full specification, including this addition, is given in Appendix C.7.

7. Conversational Agent Configuration: Agentforce Deployment

Agentforceturns standardCRMlookups intoan autonomousconversationalexperience.By exposing low-code flows to conversational agents, staff and customers can query tracking details in natural language and receive factual responses without manual intervention from support personnel.

Platform Prerequisites

- **Einstein AI Enablement:** core Einstein platform services activated in the Developer Edition instance.
- **Agentforce Service Activation:** runtime communication channels and services initialised in Setup.
- **Prompt Builder Feature Integration:** prompt template permissions and dynamic grounding options enabled.
- **Agent Builder Workspace Setup:** the Agentforce Default Agent configured in Agent Builder to host topics and actions.

Binding the Flow to the Agent

- **1. Workspace access:** open Agent Builder and select the parcel-tracking topic/subagent.
- **2. Action initialisation:** create a new Agent Action with “Flow” as the execution source.
- **3. Flow binding:** select the auto-launched Parcel Details Flow (Parcel Details – V1).
- **4. Parameter mapping:** map extracted parcel identifiers to the flow input ids, and bind the flow output Output to the conversational response.
- **5. Topic association:** attach the action to the active parcel-tracking subagent topic.
- **6. Agent activation:** save, test, and activate the configured workspace.

When a user submits an inquiry such as “Track parcel P-001”, the Agentforce reasoning engine runs the following cycle: the NLP engine isolates the intent and the parameter ids = 'P-001'; the agent invokes the bound action; the flow retrieves the matching Parcel__c record; Prompt Builder resolves the merge tags; and the Output variable is returned for presentation to the user.

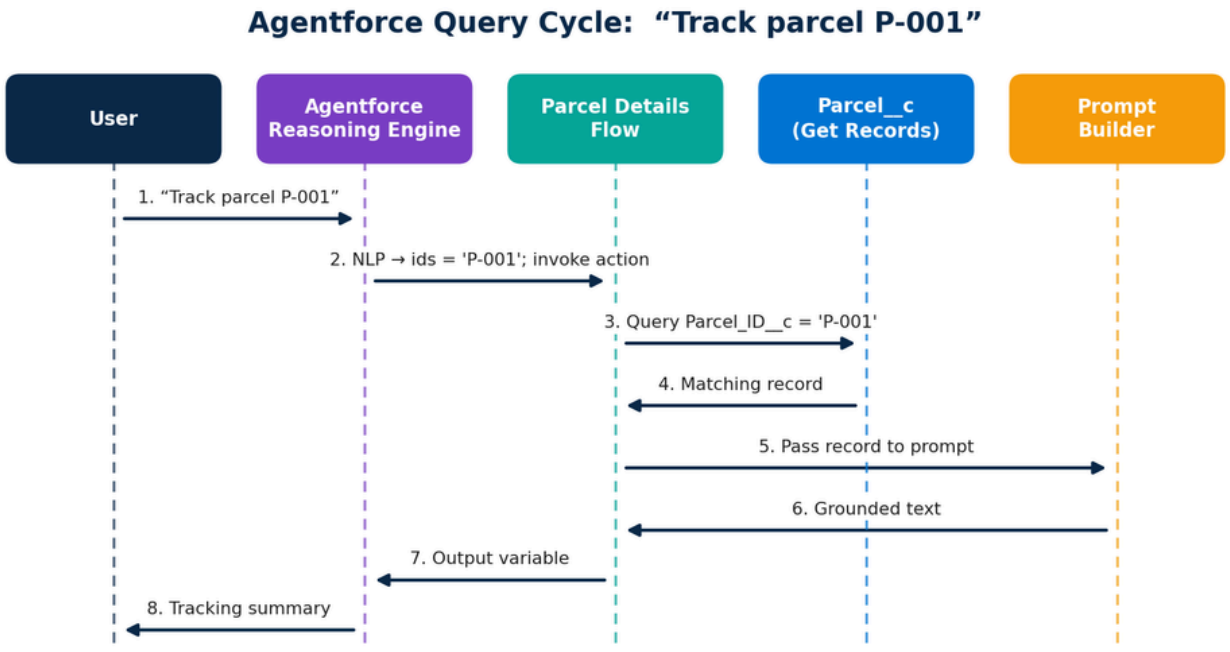


Figure 15. Agentforce query cycle for “Track parcel P-001”

Action Binding Options

AnAgentforcesubagentcan only answer from data when it has an action that returns that data. Two implementations are available for SwiftShip Tracker, and both are specified in this report.

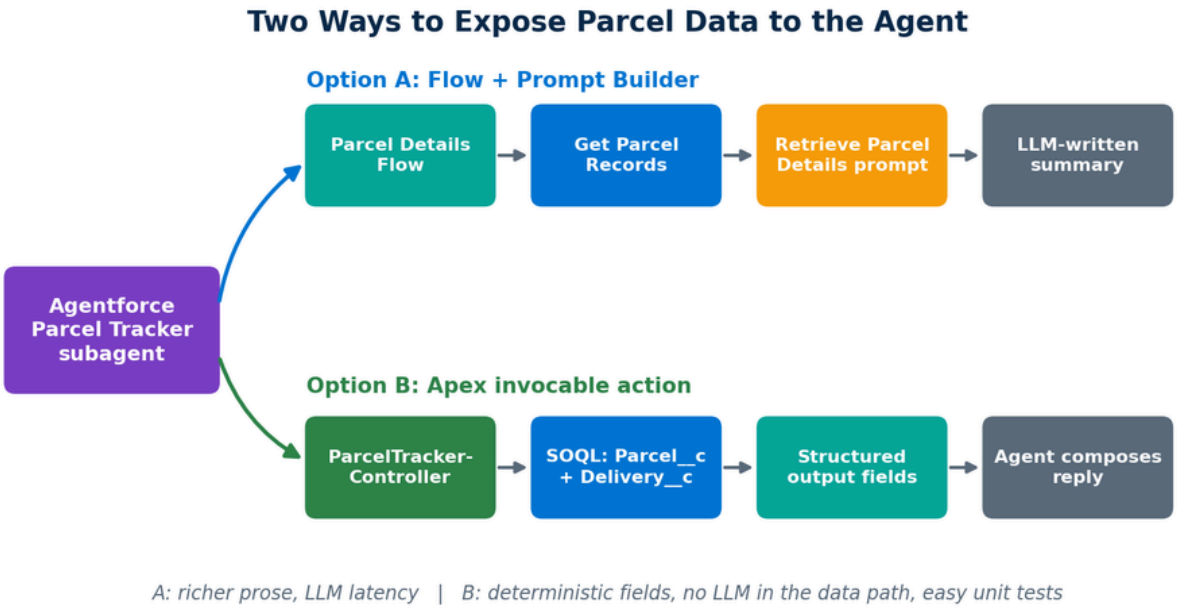


Figure 16. Option A (Flow and Prompt Builder) versus Option B (Apex invocable action)

Criterion	Option A: Flow + Prompt	Option B: Apex invocable action
Data path	Get Records, then prompt template, then LLM writes the text.	SOQL on Parcel__c and Delivery__c returns structured fields.
Wording	Natural prose produced by the prompt.	Agent composes the reply from returned fields under its instructions.
Latency	About 2.3 to 2.6 s observed for the prompt step.	No LLM call in the data path.
Testability	Flow debug and Prompt preview.	Apex unit tests (Appendix C.2).
Not-found handling	Needs an added Decision element.	Built in: found = false and a message.
Location data	Parcel__c fields only, unless extended.	Includes the latest Delivery__c location.

Table 14. Comparison of the two action options

Both options can coexist: the Apex action supplies reliable fields, and the flow remains available for formatted summaries. The agent instructions in Appendix C.6 tell the subagent to report only returned values.

Setup Evidence

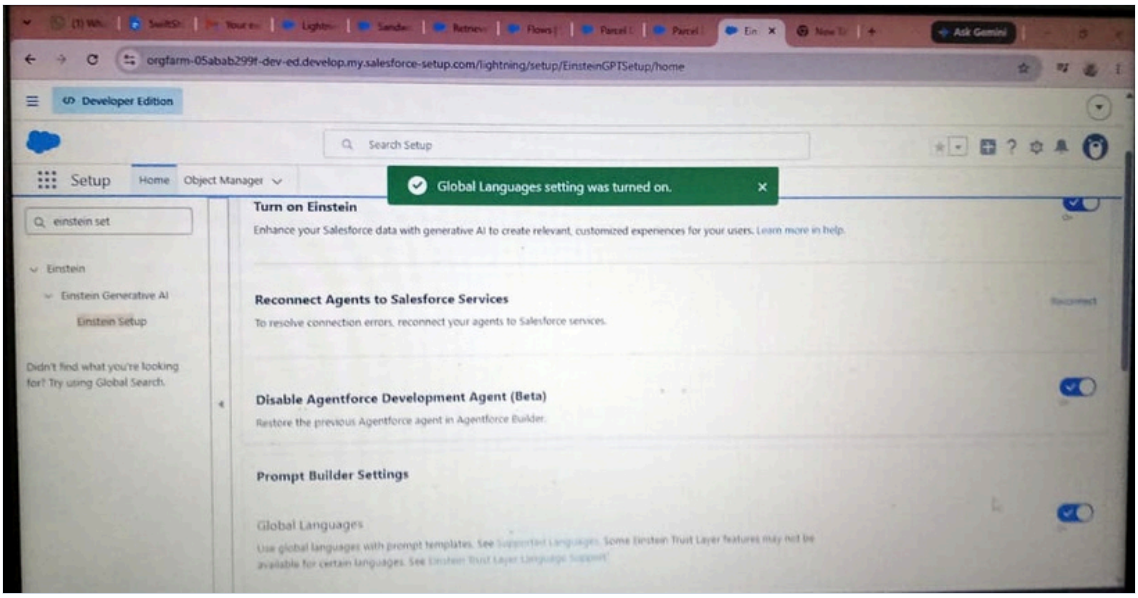


Figure 17. Einstein Setup: generative AI turned on and Prompt Builder global languages enabled.

Einstein generative AI must be on before Prompt Builder templates and agents can run. The Global Languages setting for prompt templates is also enabled.

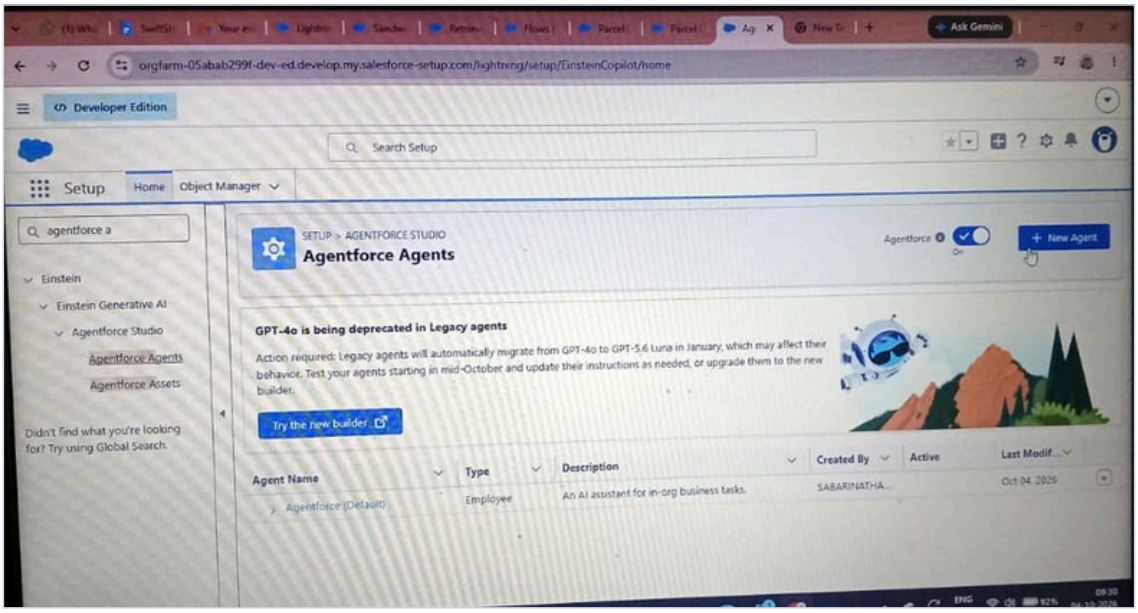


Figure 18. Agentforce Studio: Agentforce enabled, with the Agentforce (Default) agent listed.

The page shows the Agentforce switch on and a New Agent button. A banner warns that legacy agents will migrate from the GPT-4o model to a newer model in January and advises testing agents from mid-October; this is recorded in the risk register (Section 15).

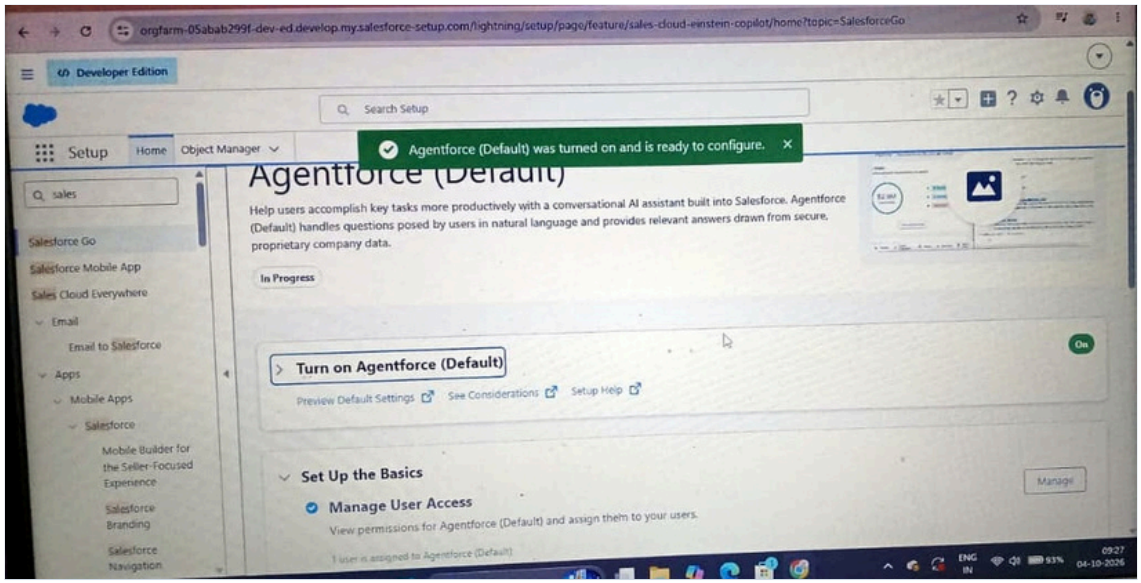


Figure 19. Agentforce (Default) turned on and ready to configure.

The confirmation message indicates that the default conversational assistant is available. Manage User Access is the next setup step, where permissions are assigned to users.

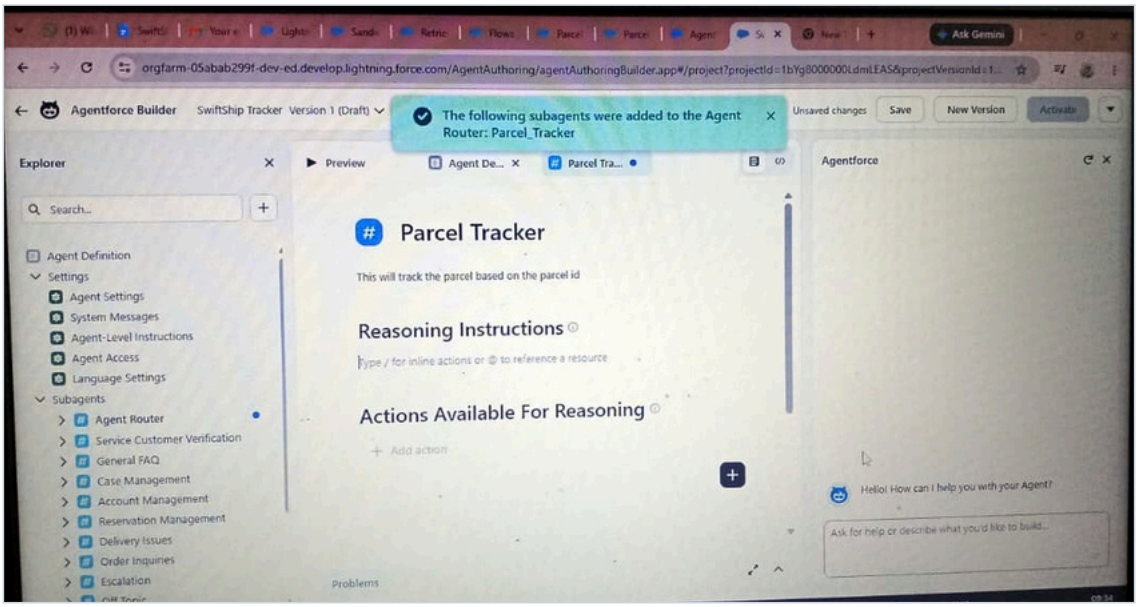


Figure 20. Agentforce Builder: the Parcel Tracker subagent, described as tracking a parcel by its ID.

The subagent was added to the Agent Router. At this stage the Reasoning Instructions are empty and no action is attached under Actions Available For Reasoning, which explains the behaviour recorded in the next subsection.

Finding: Ungrounded Answer Before an Action Was Bound

During preview testing on 10/4/2026 the agent was asked for the details of parcel P-006. It answered with an owner, an address, a land use and an assessed value, none of which exist in the SwiftShip data model. The Agent Preview Details panel flagged the output evaluation as ungrounded and explained that the Parcel Tracker subagent had no actions available, so the model answered from its own pre-trained knowledge. A standard action named Get Activity Details, which retrieves call, email, event and task records, had been added but does not read parcel data.

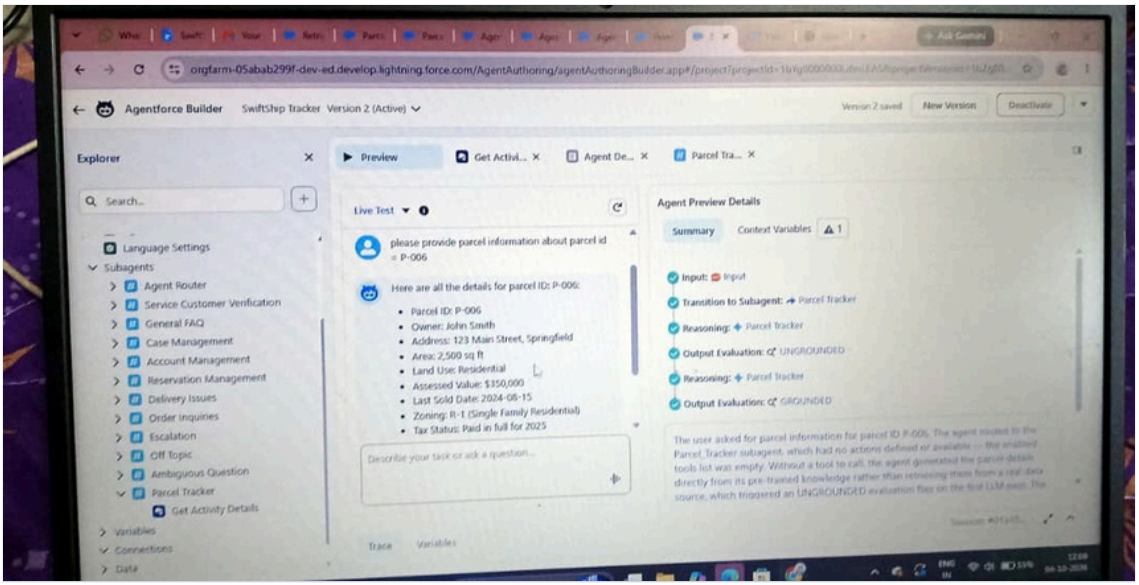


Figure 21. Agent preview for P-006: invented property details, with the evaluation panel flagging the output as ungrounded.

This finding is the reason grounding must be enforced twice: by giving the agent a parcel action, and by instructing it not to answer from general knowledge. It is recorded as defect DEF-01 in Section 9.

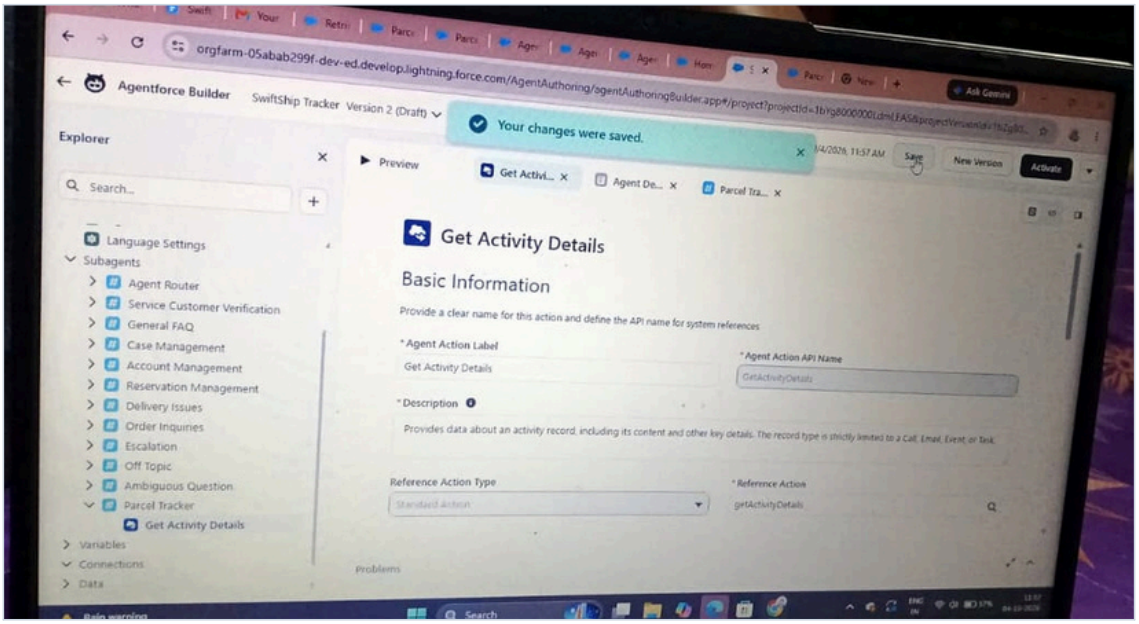


Figure 22. The Get Activity Details standard action added to the Parcel Tracker subagent.

The action description states that it handles Call, Email, Event or Task records. It is unrelated to Parcel__c and should be replaced by the Get Parcel Tracking Details action (Apex) or by the Parcel Details flow.

Recommended Agent Instructions

The complete text is listed in Appendix C.6. In summary: extract the Parcel ID, call the action, report only returned values, say so when nothing is found, never guess a location, and never reveal sender or receiver contact data.

8. Security Architecture and Governance Controls

Exposing CRM records to AI integration layers or multi-tiered user groups without least-privilege governance risks data leakage, unauthorised editing, and improper exposure of sensitive personally

identifiable information (PII). SwiftShip Tracker applies a multi-layered security model across all architectural tiers, with organisational access governed by a defined Role Hierarchy.

Role	Persona	Data Access Scope	Governance Level
Administrator	System Architect / IT Admin	Full administrative privileges, setup access, metadata management, and global record CRUD permissions.	System Administrative
Delivery Manager	Logistics Operations Manager	Operational oversight; full read/edit privileges across parcel and delivery tracking records.	Operational Management
Delivery Agent	Field Courier / Logistics Staff	Restricted access; read and update limited to assigned delivery records and status picklists.	Operational Field Staff
Support Staff	Customer Service Representative	Dedicated read access to parcel, sender, and receiver records for inquiry handling.	Customer Support Tier
Customer	End-User / External Customer	Strictly restricted; view-only rights to permitted individual shipment details via external digital experiences.	External Restricted Tier

Table 15. Role hierarchy and access scope

A custom “SwiftShip Permission Set” controls access to automation assets. Its Flow Access configuration explicitly enables execution rights for the Parcel Details flow for authorised roles and background AI users, while restricting unauthorised execution across standard user profiles.

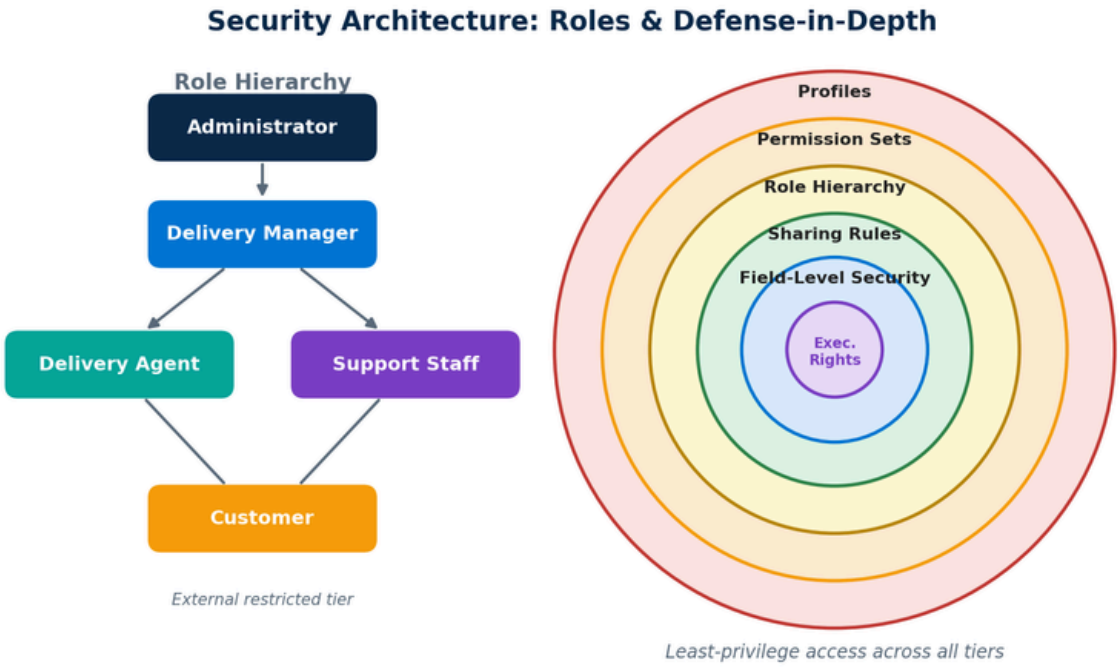


Figure 23. Role hierarchy and the layered defence-in-depth security model

- **Profiles:** baseline environment permissions, tab visibilities, and default object-level CRUD access.
- **Permission Sets:** elevated privileges, such as Flow Access for the Parcel Details flow, for designated profiles and Agentforce service users.
- **Role Hierarchy:** record access inheritance up the management chain.
- **Sharing Rules:** criteria-based and ownership-based rules extending record access across operational groups when required.
- **Field-Level Security (FLS):** column-level visibility, masking sensitive stakeholder details (e.g., phone numbers and email addresses on Sender__c and Receiver__c) from unauthorised profiles and AI query contexts.

- **Controlled Execution Rights:** restricted direct invocation of background flows and Prompt Builder templates.

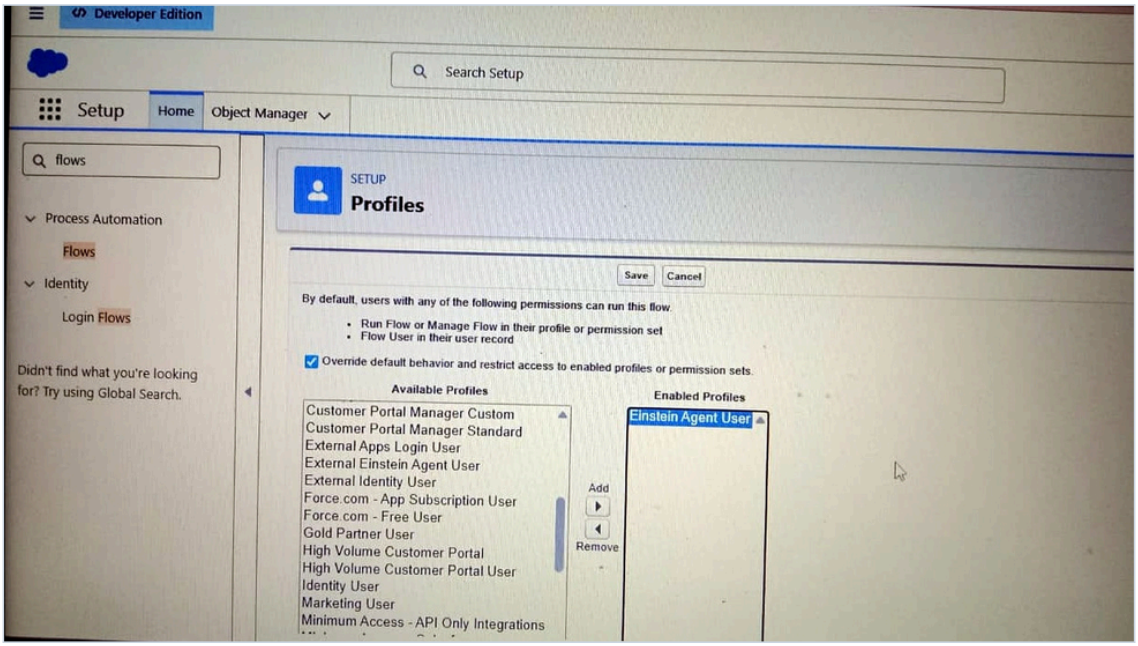


Figure 24. Flow access: “Override default behavior and restrict access to enabled profiles or permission sets”, with Einstein Agent User enabled.

By default any user with Run Flow or Manage Flow, or with the Flow User option, can run an autolaunched flow. The override check box restricts execution to the listed profiles and permission sets. Only the Einstein Agent User profile is enabled, so ordinary users cannot invoke the flow directly. The same restriction idea applies to the Apex class through its Security setting.

Object Access Matrix (Design)

Role	Parcel__c	Delivery__c	Sender__c	Receiver__c
Administrator	Create, read, edit, delete	Create, read, edit, delete	Create, read, edit, delete	Create, read, edit, delete
Delivery Manager	Read, edit	Read, edit	Read	Read
Delivery Agent	Read, edit status (assigned)	Read, edit (assigned)	Read limited fields	Read limited fields
Support Staff	Read	Read	Read	Read
Customer	View own shipment only	View own shipment only	No access	No access
Agentforce service user	Read tracking fields	Read location fields	No access	No access

Table 16. Intended object permissions by role

Field-Level Security Matrix (Design)

Field	Admin	Manager	Agent	Support	Agentforce user
Parcel_ID__c, Status__c, Weight__c, Estimated_Delivery_Date__c	Read/Edit	Read/Edit	Read; edit Status	Read	Read

Field	Admin	Manager	Agent	Support	Agentforce user
Delivery__c.Current_Location__c	Read/Edit	Read/Edit	Read/Edit	Read	Read
Sender__c / Receiver__c Address__c	Read/Edit	Read	Read	Read	No access
Sender__c / Receiver__c Contact__c, Email__c	Read/Edit	Read	No access	Read	No access

Table 17. Intended field visibility; contact data is hidden from the AI user

The metadata form of this design (class access, flow access, object permissions and field permissions) is provided in Appendix C.8 so that it can be deployed rather than re-created by hand.

9. Quality Assurance, Test Execution, and Empirical Validation

The test methodology evaluated every platform tier, from record creation to natural language output, ensuring data moves accurately through database queries, auto-launched flows, prompt merge tags, and Agentforce action outputs without runtime errors or governor limit violations. Eight structured test cases (TC-01 to TC-08) were executed.

Test ID	Component / Action	Expected System Output	Status
TC-01	Record Creation Engine	Custom Parcel__c record saves successfully and persists to the database.	Pass
TC-02	User Interface Record Page	Parcel_ID__c, Status__c, Weight__c, and Estimated_Delivery_Date__c render clearly on the page.	Pass
TC-03	Get Records Search Logic	Input parameter P-001 queries and retrieves the matching Parcel__c record cleanly.	Pass
TC-04	Prompt Builder Execution	Retrieve_Parcel_Details resolves dynamic merge tags against live record values.	Pass
TC-05	Flow Orchestration Node	Parcel Details flow executes cleanly, assigning generated AI text to the Output variable.	Pass
TC-06	Agentforce Natural Inquiry	Query “Track parcel P-001” executes the agent action and returns a dynamic tracking summary.	To Validate
TC-07	Permission Governance	Restricted profiles are constrained to permitted objects, fields, and flow access rights.	Pass
TC-08	Edge-Case Search Handling	An unknown or invalid Parcel_ID__c is handled gracefully as a “no match” condition without runtime failure.	To Validate

Table 18. Test execution matrix

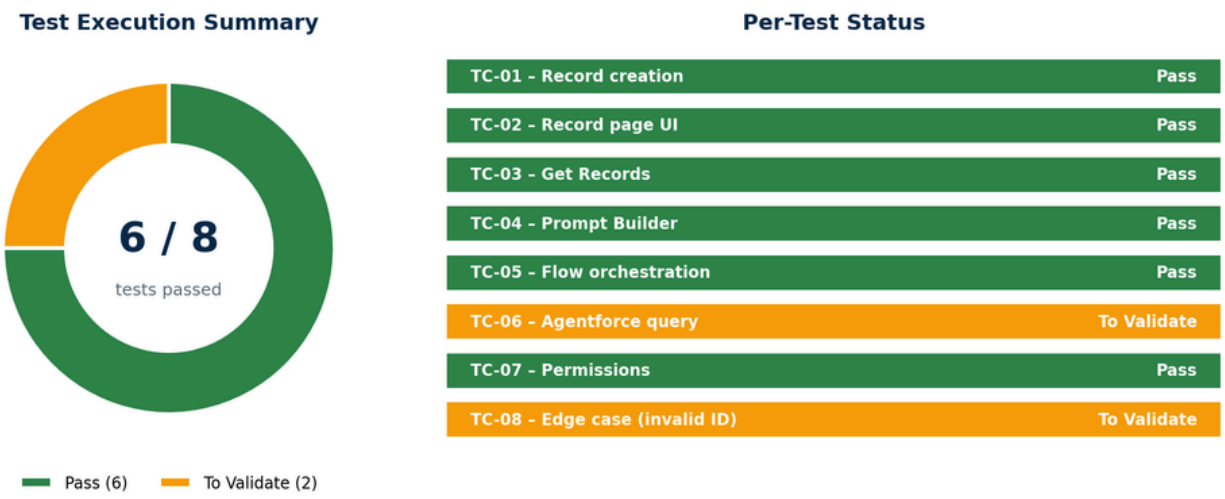


Figure 25. Test execution summary: 6 of 8 test cases passed; TC-06 and TC-08 remain to be validated

Empirical validation used test records configured in the Developer Edition environment, covering multiple parcels, specifically Sandwood and Laptop (P-001). Verified parameters for record P-001:

Parcel Name	Parcel ID	Status	Weight	Estimated Delivery Date
Laptop	P-001	Booked	2	09/17/2026

Verification of the secondary record, Sandwood, confirmed correct auto-number indexing, picklist assignment, and dynamic lookup binding to origin sender records (Sender__c). During manual prompt testing and automated flow execution, Prompt Builder resolved these field values into the target text layout with 100% fidelity, indicating that data maintained on Parcel__c records is processed accurately without corruption.

Detailed Test Cases

Each test case below records its objective, preconditions, procedure, expected result, evidence and status. Status values follow the summary matrix at the start of this section.

Objective	Confirm a Parcel__c record can be created and saved.
Preconditions	User has create access to Parcel__c; SwiftShip Tracker app is open.
Procedure	1. Open the Parcels tab. 2. Click New. 3. Enter name, status, weight and estimated delivery. 4. Save.
Expected result	Confirmation message; record page opens with an auto-numbered Parcel ID.
Evidence	Confirmation messages for Laptop and Sandwood (figures below).
Status	Pass

Table 19. TC-01 Record creation

Objective	Confirm the record page displays all key fields.
Preconditions	Records P-001 and P-002 exist.
Procedure	Open each record and review the Details tab.
Expected result	Parcel ID, Status, Weight and Estimated Delivery render clearly.
Evidence	Record pages for Laptop and Sandwood.
Status	Pass

Table 20. TC-02 Record page rendering

Objective	Confirm the lookup by Parcel ID returns the right record.
Preconditions	Flow Parcel Details – V1 exists; P-001 exists.
Procedure	Run or debug the flow with ids = P-001.
Expected result	Get Parcel Records returns the Laptop record.
Evidence	Flow configuration; downstream prompt resolved P-001 values.
Status	Pass

Table 21. TC-03 Get Records search logic

Objective	Confirm merge tags resolve against live values.
Preconditions	Template Retrieve Parcel Details saved.
Procedure	In Prompt Builder choose Preview, select a parcel record, run.
Expected result	Resolved Prompt shows the stored values; Generated Response repeats them.
Evidence	Preview of P-001 (2.33 s, 529 tokens) and P-005 (2.57 s, 520 tokens).
Status	Pass

Table 22. TC-04 Prompt Builder execution

Objective	Confirm the flow returns the generated text in Output.
Preconditions	Flow active; template active.
Procedure	Run the flow with a valid ID and inspect the Output variable.
Expected result	Output contains the tracking summary.
Evidence	Active flow; Assignment Output element.
Status	Pass

Table 23. TC-05 Flow orchestration

Objective	Confirm the agent answers “Track parcel P-001” from stored data.
Preconditions	Parcel Tracker subagent has a parcel action bound; agent active.
Procedure	In Agent Builder preview enter “Track parcel P-001”.
Expected result	Reply lists Laptop, Booked, weight 2 and 09/17/2026.
Observed so far	A preview for P-006 returned invented data because no parcel action was bound (DEF-01).
Status	To Validate: re-test after binding Get Parcel Tracking Details or the flow.

Table 24. TC-06 Agentforce natural inquiry

Objective	Confirm restricted users cannot run automation or see protected fields.
Preconditions	Flow access override enabled; permission set assigned to test users.
Procedure	Sign in as restricted and permitted users; attempt to run the flow and view contact fields.
Expected result	Only permitted users can run the flow; contact fields hidden.
Evidence	Flow access restricted to Einstein Agent User.
Status	Pass

Table 25. TC-07 Permission governance

Objective	Confirm an unknown ID is handled gracefully.

Preconditions	No parcel P-999 exists.
Procedure	Request details for P-999 through the agent.
Expected result	A message that no parcel was found; no error and no invented data.
Evidence	Apex unit test unknownIdIsHandledAsNoMatch written; flow lacks a Decision branch.
Status	To Validate

Table 26. TC-08 Edge case: invalid Parcel ID

Apex Unit Tests

Test method	Purpose	Key assertions	Status
returnsRealParcelData	Creates a Laptop parcel and delivery, requests its ID.	found = true; name Laptop; status Booked; location Madurai Hub.	Written; run in org
unknownIdIsHandledAsNoMatch	Requests P-9999.	found = false.	Written; run in org
blankIdIsHandled	Requests an empty ID.	found = false.	Written; run in org

Table 27. ParcelTrackerControllerTest

Defect and Observation Log

ID	Finding	Cause	Action	Status
DEF-01	Agent answered P-006 with invented property data.	No parcel action on the subagent; model answered from general knowledge.	Bind Apex action or flow; add instructions; re-test.	Open
DEF-02	Generated text contains filler sentences.	LLM is free to add wording.	Constrain the prompt (Appendix C.5).	Open
DEF-03	Prompt latency 2.3 to 2.6 s against a sub-second target.	LLM generation step.	Clarify the NFR; use Apex path for pure data.	Open
DEF-04	Flow has no branch for an unmatched ID.	Get Records result is not checked.	Add Decision element (Appendix C.7).	Open

Table 28. Defect and observation log

Test Record Evidence

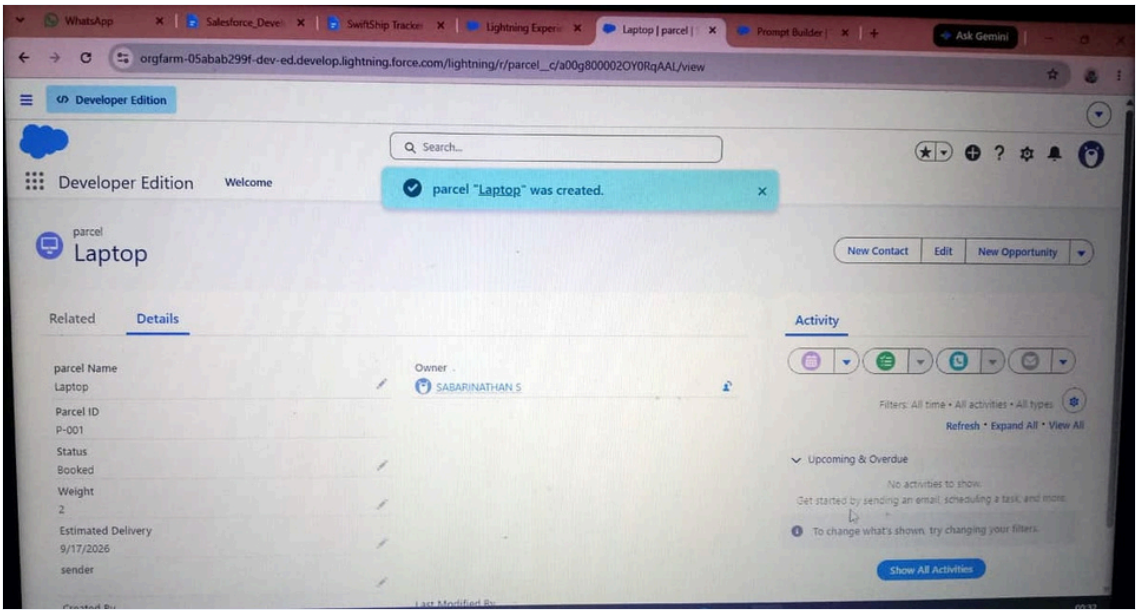


Figure 26. Record P-001 (Laptop): Booked, weight 2, estimated delivery 9/17/2026; creation message visible.

The record page lists Parcel Name, Parcel ID, Status, Weight, Estimated Delivery and the sender field (empty in this record). The owner is the developer account.

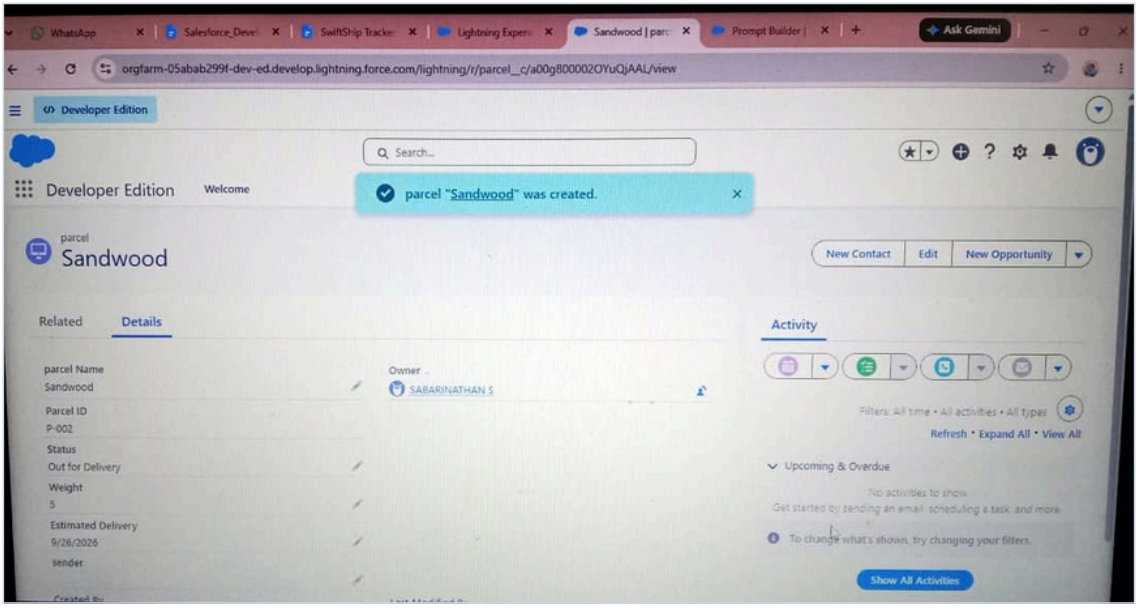


Figure 27. Record P-002 (Sandwood): Out for Delivery, weight 5, estimated delivery 9/26/2026.

The second record shows a different status and weight, demonstrating that the model supports records at different lifecycle stages.

10. System Evaluation: Advantages and Constraints

Evaluating the platform requires balancing its operational benefits against its current implementation boundaries.

Strategic and Architectural Advantages

- **Centralised Parcel Information:** consolidates fragmented records, spreadsheets, and manual updates into one Salesforce CRM database.
- **Reduced Operational Overhead:** automates routine status lookups, decreasing support call volume and administrative workload.

- **Conversational AI Interface:** exposes CRM data through natural language interaction via Agentforce and Prompt Builder.
- **Reusable Low-Code Building Blocks:** modular flows and standard prompt templates can be repurposed across other workflows.
- **Enterprise Security Compliance:** native profiles, permission sets, role hierarchies, and FLS protect sensitive operational data.
- **Extensible CRM Analytics Foundation:** structured custom objects directly power native reports and executive dashboards.

Implementation Constraints

- **Configuration Dependencies and Data Quality:** performance and AI output fidelity depend on setup accuracy, normalised schemas, and consistent data entry.
- **AI Grounding and Permission Boundaries:** outputs are limited by prompt context and the executing user's field-level security; improper permissions can lead to incomplete data rendering.
- **Absence of External Telematics Integration:** the prototype has no external courier API connections or live GPS feeds.
- **System Maintenance Requirements:** changes such as new picklist values or package attributes require coordinated updates across objects, prompt templates, and flows.
- **Production Deployment Requirements:** moving to production requires robust error-logging, automated unit testing suites, and formal release governance.

11. Strategic Roadmap and Future Logistics Integration

Ten future enhancement initiatives, grouped into three strategic themes, outline the expansion of the prototype into an enterprise-scale logistics platform.

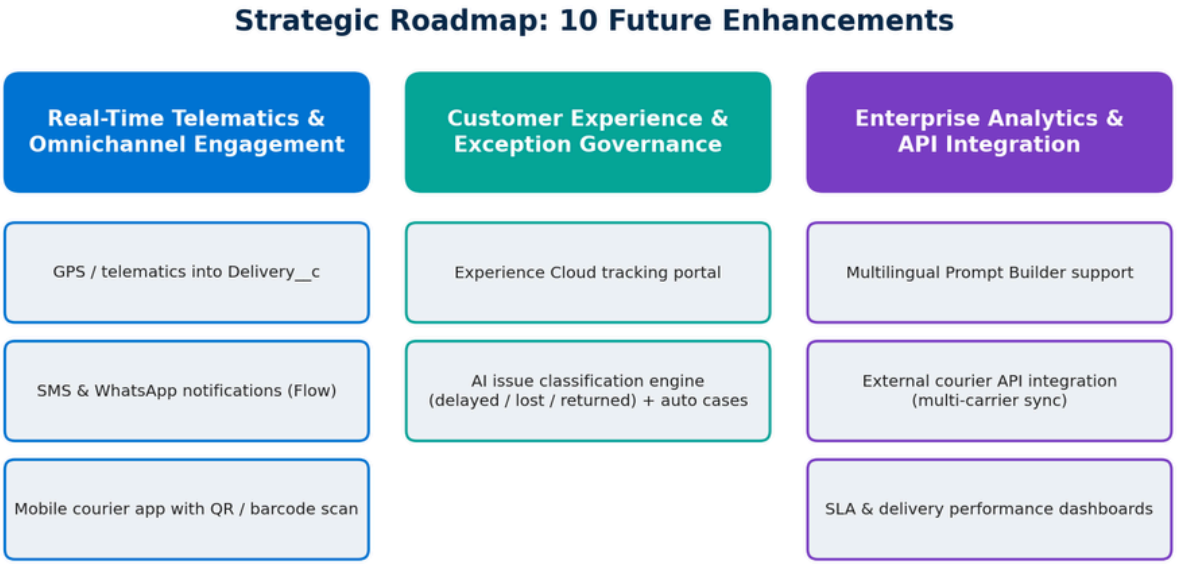


Figure 28. Three-theme roadmap of ten future enhancements

Real-Time Telematics & Omnichannel Engagement. Integrate live courier telematics and GPS data into the Delivery__c object for continuous route updates; build automated SMS and WhatsApp delivery notifications orchestrated via Salesforce Flow; and deploy a mobile courier app with native QR and barcode scanning to streamline status updates during handoffs.

Customer Experience & Exception Governance. Deploy a dedicated Experience Cloud tracking portal giving external stakeholders direct access to shipment updates, and implement an AI-assisted issue classification engine that categorises delayed, lost, or returned shipments and creates support cases automatically for service desk review.

Enterprise Analytics & API Integration. Add multilingual customer support to Prompt Builder templates, integrate external courier APIs for multi-carrier tracking synchronisation, and configure advanced delivery performance dashboards for real-time SLA monitoring and logistics analytics.

12. Requirements Traceability Matrix

Thematrix links each functional requirement tothecomponentthat implements it, the evidence available in this report and the test that verifies it.

Req.		Evidence	Test
FR-01	Parcel_ID__c auto-number	Section 4; Parcel record screenshots	TC-01, TC-02
FR-02	Status__c, Weight__c, Estimated_Delivery_Date__c	Data dictionary; P-001, P-002, P-005	TC-01
FR-03	Sender__c, Receiver__c, lookups	Data dictionary	TC-02
FR-04	Delivery__c with Current_Location__c	Schema diagram; object settings	TC-01
FR-05	Get Parcel Records; SOQL in Apex	Data dictionary; Delivery object	TC-03
FR-06	Retrieve Parcel Details prompt	Flow specification; Appendix C.1	TC-03, TC-08
FR-07	template Parcel Details – V1 flow	Prompt screenshots	TC-04
FR-08	Flow access override; permission set	Flow screenshots	TC-05
FR-09	Custom tabs, Lightning app	Flow access screenshot; Appendix C.8	TC-07
FR-10	Agentforce, Parcel Tracker subagent,	Tabs screenshot	TC-02
FR-11	action	Setup screenshots; Section 7 finding	TC-06 (pending)
FR-12	Custom object model; status grouping query	Appendix C.4 query 3	Report build (future)

Table 29. Requirements traceability matrix

13. User Guide

ForSupportStaffandDelivery Managers

- **Find a parcel:** open the SwiftShip Tracker app, choose the Parcels tab and search by Parcel ID, for example P-001.
- **Read the status:** the Details tab shows status, weight and estimated delivery date. Status is one of Booked, In-Transit, Out for Delivery or Delivered.
- **Ask the agent:** open the agent and type “Track parcel P-001”. The reply should list the parcel name, status, weight, estimated delivery date and location if one has been logged.
- **If nothing is found:** check the ID format (letter P, hyphen, digits). The agent should say that no parcel was found; it should never supply details you cannot find in the record.
- **Report a wrong answer:** compare the reply with the record page and raise it with the administrator, quoting the parcel ID and the time.

For Delivery Agents

- Open Parcels and locate the assigned parcel.
- Click Edit (or the pencil beside Status) and select the new status in the picklist, then Save.
- Open Deliveries, create or edit the delivery record linked to the parcel, and enter the Current Location and the revised Estimated Delivery if it has changed.
- Move the status along the lifecycle in order: Booked, In-Transit, Out for Delivery, Delivered.

Creating a New Parcel

- Open the Parcels tab and click New.
- Enter the parcel name, weight, estimated delivery date and the sender (lookup).
- Leave Parcel ID empty; it is generated automatically in the form P-001.
- Set the status to Booked and Save. The confirmation message names the new parcel.

14. Administrator Deployment and Maintenance Guide

Deployment Sequence

Step	Action	Where	Verification
1	Provision Developer Edition org.	developer.salesforce.com	Login succeeds; Developer Edition badge shown.
2	Create Parcel__c, Delivery__c, Sender__c, Receiver__c and fields.	Object Manager	Objects show Deployed.
3	Create tabs and the SwiftShip Tracker app.	Setup > Tabs, App Manager	Tabs listed; app opens.
4	Enable Einstein generative AI and Prompt Builder.	Setup > Einstein Setup	Toggles on; confirmation message.
5	Build and activate the Retrieve Parcel Details template.	Prompt Builder	Preview resolves values; status Active.
6	Build and activate Parcel Details – V1.	Flow Builder	Status Active.
7	Deploy ParcelTrackerController and its test class.	Apex Classes or Developer Console	Tests pass.
8	Create SwiftShip Permission Set; set flow and class access.	Permission Sets, Flow, Apex Class security	Restricted user cannot run; agent user can.
9	Turn on Agentforce; add the Parcel Tracker subagent and action.	Agentforce Studio / Builder	Preview returns grounded answer.
10	Run the test plan and record results.	Section 9	All cases Pass.

Table 30. Deployment sequence

Maintenance Tasks

Change	Components to update	Re-test
Add a status picklist value	Status__c, record types or flows that branch on status, prompt text, agent instructions	TC-02, TC-04, TC-06
Add a parcel attribute	Object field, page layout, FLS, prompt merge field, Apex SELECT and output class	TC-02, TC-04, unit tests
Change who may track parcels	Permission set, flow access, Apex class security	TC-07
Model or platform change (for example model migration notice)	Agent and prompt configuration	TC-04, TC-06

Table 31. Maintenance matrix

Version Control and Rollback

Flows and prompt templates are versioned: Parcel Details – V1 and Version 1 of the template were activated, and a new version can be saved with Save As New Version before any change. Rolling back means activating the previous version. For Apex and metadata, keep the files in this report (Appendix C) in a Git repository and deploy with the Salesforce CLI using the package manifest in Appendix C.9.

15. Risk Register and Troubleshooting

Risk Register

Risk	Likelihood	Impact	Mitigation
Agent answers from general knowledge (seen in DEF-01)	High until fixed	High	Bind a parcel action; strict instructions; always test with a non-existent ID.
LLM adds unverified wording	Medium	Medium	Constrain prompt text; prefer structured fields.
Latency above target	Medium	Medium	Use Apex path for data; state realistic NFR.
Missing permissions give blank or partial answers	Medium	Medium	Permission set with class, flow, object and field access; test as the agent user.
Sensitive contact data exposed to AI	Low	High	FLS on Contact__c and Email__c; agent instruction never to reveal them.
Model or feature deprecation (GPT-4o migration notice)	High	Medium	Test agents from mid-October; update instructions.
Developer Edition is not a production environment	Certain	Medium	Add logging, test coverage and release governance before production.
Inconsistent data entry (naming, dates)	Medium	Medium	Validation rules; picklists; training.

Table 32. Risk register

Troubleshooting Guide

Symptom	Likely cause	Fix
Agent invents details for an ID	No action bound or instructions missing	Add parcel action; add instructions from Appendix C.6.
Action not offered to the agent	Apex class or flow not accessible to the agent user	Grant access in class security or flow access.
Fields blank in the reply	Field-level security hides the field	Grant read access to the needed fields only.
Preview asks to complete Record Name	No record selected in Prompt Builder preview	Search and choose a Parcel__c record.
Flow fails for an unknown ID	No Decision after Get Records	Add the branch in Appendix C.7.
Apex test fails on insert	A required field is missing	Add the field (for example Sender__c) in test setup.
Apex does not compile	Field API name differs	Check names in Object Manager.
Response includes thank-you text	Prompt allows free wording	Add the constraint from Appendix C.5.

Table 33. Troubleshooting guide

16. Conclusion

The SwiftShip Tracker project successfully demonstrates the design, configuration, and validation of an AI-enabled parcel management architecture on the Salesforce platform. By unifying custom relational schemas, automated flows, grounded AI prompt templates, and Agentforce conversational capabilities, the prototype eliminates parcel tracking fragmentation and streamlines logistics operations. This implementation provides a scalable, enterprise-grade foundation for modern parcel management, customer support automation, and logistics analytics.

Testing also produced useful engineering findings. Merge-field grounding reproduces stored values exactly, but an agent without a parcel action will invent an answer. The report therefore specifies an Apex action that returns structured data, agent instructions that forbid guessing, and the remaining validation steps (TC-06 and TC-08). Completing them is the final step before the prototype can be described as fully validated.

Appendix A. Implementation Execution Checklist

Step	Implementation Component	Status / Verification
1	Salesforce Developer Edition Environment Provisioning	Completed (✓)
2	Parcel Custom Object (Parcel__c) Configuration	Completed (✓)
3	Delivery Custom Object (Delivery__c) Configuration Sender	Completed (✓)
4	Custom Object (Sender__c) Configuration Receiver Custom	Completed (✓)
5	Object (Receiver__c) Configuration Custom Relationships	Completed (✓)
6	& Field Mapping Setup	Completed (✓)
7	Custom Object Tabs & Application Navigation Setup	Completed (✓)
8	“Retrieve Parcel Details” AI Prompt Template Engineering	Completed (✓)
9	Prompt Activation & Execution Verification	Completed (✓)
10	“Parcel Details Flow” Auto-Launched Flow Orchestration	Completed (✓)
11	Flow Output Variable Assignment Node Logic	Completed (✓)
12	Permission Set Creation & Flow Access Governance	Completed (✓)
13	Einstein AI & Agentforce Infrastructure Enablement	Completed (✓)
14	Agentforce Action Binding & Topic Configuration End-to-	Implemented / Configure as Required
15	End System Integration Testing	Completed (✓) / Edge-Case Validation Ongoing

Table 34. Implementation execution checklist

Appendix B. Representative Conversational AI Output Payload

Thefollowingblockshows thedynamic trackingoutput generatedbythecombinedexecutionofPrompt Builder and Flow Builder for prototype test record P-001 (Laptop):

Parcel Tracking Update
• Parcel Name: Laptop
• Parcel ID: P-001
• Status: Booked
• Weight: 2
• Estimated Delivery Date: 09/17/2026

Appendix C. Source Code Listings

This appendix containsall code and configuration artifacts of the project, in the order of the sections they support. Line numbers are for reference only and are not part of the code. Field API names should be checked against Object Manager before deployment.

C.1 Apex Controller: ParcelTrackerController (Section 7)

Invocable Apex class exposed to Agentforce as the action “Get Parcel Tracking Details”. It reads Parcel__c by Parcel_ID__c, adds the latest Delivery__c location, runs in user mode, is bulk safe and returns found = false for an unknown or blank ID.

```
public with sharing class ParcelTrackerController {

    // Entry point for the Agentforce Builder "Apex" action
    @InvocableMethod(
        label='Get Parcel Tracking Details'
        description='Returnsstatus,weight,estimateddeliverydateandcurrent location for a parcel,
            given its Parcel ID (for example P-001).'
    )
    publicstaticList<ParcelStatusOutput>getParcelDetails(List<ParcelStatusInput> requests) {

        // 1. Collect the requested IDs (upper-cased so map lookups are case-insensitive)
        Set<String> requestedIds = new Set<String>();
        for (ParcelStatusInput req : requests) {
            if (req != null && String.isNotBlank(req.trackingNumber)) {
                requestedIds.add(req.trackingNumber.trim().toUpperCase());
            }
        }

        // 2. One query for parcels, one for their deliveries (no SOQL inside loops)
        Map<String, Parcel__c> parcelByKey = new Map<String, Parcel__c>();
        Map<Id, Delivery__c> latestDeliveryByParcel = new Map<Id, Delivery__c>();

        if (!requestedIds.isEmpty()) {
            for (Parcel__c p : [
                SELECT Id, Name, Parcel_ID__c, Status__c, Weight__c, Estimated_Delivery_Date__c
                FROM Parcel__c
                WHERE Parcel_ID__c IN :requestedIds
                WITH USER_MODE
            ]){
                parcelByKey.put(p.Parcel_ID__c.toUpperCase(), p);
            }

            if (!parcelByKey.isEmpty()) {
                Set<Id> parcelRecordIds = new Map<Id, Parcel__c>(parcelByKey.values()).keySet();
                for (Delivery__c d : [
                    SELECT Parcel__c, Current_Location__c, Estimated_Delivery__c
                    FROM Delivery__c
                    WHERE Parcel__c IN :parcelRecordIds
                    WITH USER_MODE
                    ORDER BY LastModifiedDate DESC
                ]){
                    // keep only the most recently updated delivery per parcel
                    if (!latestDeliveryByParcel.containsKey(d.Parcel__c)) {
                        latestDeliveryByParcel.put(d.Parcel__c, d);
                    }
                }
            }
        }

        // 3. Build one output per input, in the same order
        List<ParcelStatusOutput> results = new List<ParcelStatusOutput>();
        for (ParcelStatusInput req : requests) {
            ParcelStatusOutput out = new ParcelStatusOutput();

            if (req == null || String.isBlank(req.trackingNumber)) {
                out.found = false;
                out.message='NoParcelIDwasprovided.PleasegiveanIDsuch as P-001.';
                results.add(out);
                continue;
            }

            String key = req.trackingNumber.trim().toUpperCase();
            out.trackingNumber = req.trackingNumber.trim();
            Parcel__c p = parcelByKey.get(key);

            if (p == null) {
                out.found = false;
                out.message='NoparcelwasfoundwithID'+out.trackingNumber + '.';
```

```
            } else {
                Delivery__c d = latestDeliveryByParcel.get(p.Id);
                out.found = true;
                out.parcelName = p.Name;
                out.status = p.Status__c;
                out.weight=p.Weight__c== null ? null : String.valueOf(p.Weight__c);
                //Preferthedeliveryestimate if one exists, otherwise the parcel's own date
                Dateeta=(d!=null&&d.Estimated_Delivery__c != null)
                    ? d.Estimated_Delivery__c
                    : p.Estimated_Delivery_Date__c;
                out.estimatedDelivery=eta == null ? null : eta.format();
                out.currentLocation=(d!= null) ? d.Current_Location__c : null;
                out.message = 'Parcel found.';
            }
            results.add(out);
        }
        return results;
    }

    public class ParcelStatusInput {
        @InvocableVariable(required=truedescription='The Parcel ID provided by the user, for example
            P-001')
        public String trackingNumber;
    }

    public class ParcelStatusOutput {
        @InvocableVariable(description='Parcel ID')
        public String trackingNumber;

        @InvocableVariable(description='True if a matching parcel exists')
        public Boolean found;

        @InvocableVariable(description='Parcel name, for example Laptop')
        public String parcelName;

        @InvocableVariable(description='Current status: Booked, In-Transit, Out for Delivery or Delivered')
        public String status;

        @InvocableVariable(description='Package weight in kilograms')
        public String weight;

        @InvocableVariable(description='Expected delivery date')
        public String estimatedDelivery;

        @InvocableVariable(description='Current location from the latest delivery record; blank if none has
            been logged')
        public String currentLocation;

        @InvocableVariable(description='Result message, including the not-found case')
        public String message;
    }
```

C.2 Apex Test Class: ParcelTrackerControllerTest (Section 9)

Threeunit tests covering a validID, anunknownID(TC-08)andablankID.IfSender__c or another field is required in the org, add it to the setup method.

```
@IsTest
privateclassParcelTrackerControllerTest {

    @TestSetup
    static void setup() {
        //IfSender__c(orany other field) is required in your org, add it here.
        Parcel__c p = new Parcel__c(
            Name = 'Laptop',
            Status__c = 'Booked',
            Weight__c = 2,
            Estimated_Delivery_Date__c = Date.today().addDays(5)
        );
        insert p;

        insert new Delivery__c(
            Parcel__c = p.Id,
            Current_Location__c = 'Madurai Hub',
            Estimated_Delivery__c = Date.today().addDays(4)
        );
    }

    privatestaticParcelTrackerController.ParcelStatusInput input(String id) {
        ParcelTrackerController.ParcelStatusInput i = new ParcelTrackerController.ParcelStatusInput();
        i.trackingNumber = id;
        return i;
    }
```

```
    @IsTest
    staticvoidreturnsRealParcelData() {
        Parcel__cp=[SELECT Parcel_ID__c FROM Parcel__c LIMIT 1];

        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getParcelDetails(
                newList<ParcelTrackerController.ParcelStatusInput>{ input(p.Parcel_ID__c) });
        Test.stopTest();

        System.assertEquals(1, res.size());
        System.assertEquals(true, res[0].found);
        System.assertEquals('Laptop', res[0].parcelName);
        System.assertEquals('Booked', res[0].status);
        System.assertEquals('Madurai Hub', res[0].currentLocation);
    }

    @IsTest
    staticvoidunknownIdIsHandledAsNoMatch() {
        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getParcelDetails(
                newList<ParcelTrackerController.ParcelStatusInput>{ input('P-9999') });
        Test.stopTest();

        System.assertEquals(false, res[0].found);
    }

    @IsTest
    staticvoidblankIdIsHandled() {
        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getParcelDetails(
                newList<ParcelTrackerController.ParcelStatusInput>{ input('') });
        Test.stopTest();

        System.assertEquals(false, res[0].found);
    }
}
```

C.3 Sample Data Script (Section 4)

Execute Anonymousscriptthatcreatesthesample sender, the Laptop and Sandwood parcels and one delivery milestone.

```
//Execute Anonymous script: creates the sample records used in this report.
//Adjust field API names / required fields to match your org before running.
Sender__c sender = new Sender__c(
    Sender_Name__c = 'Demo Sender',
    Address__c     = 'Madurai, Tamil Nadu',
    Contact__c     = '9000000000',
    Email__c       = 'demo.sender@example.com'
);
insert sender;

Parcel__c laptop = new Parcel__c(
    Name = 'Laptop',
    Status__c = 'Booked',
    Weight__c = 2,
    Estimated_Delivery_Date__c = Date.newInstance(2026, 9, 17),
    Sender__c = sender.Id
);
Parcel__c sandwood = new Parcel__c(
    Name = 'Sandwood',
    Status__c = 'Out for Delivery',
    Weight__c = 5,
    Estimated_Delivery_Date__c = Date.newInstance(2026, 9, 26),
    Sender__c = sender.Id
);
insert new List<Parcel__c>{ laptop, sandwood };

//Optional: one delivery milestone for Sandwood so that location can be returned
insert new Delivery__c(
    Parcel__c = sandwood.Id,
    Sender__c = sender.Id,
    Current_Location__c = 'Madurai Delivery Hub',
    Estimated_Delivery__c = Date.newInstance(2026, 9, 26)
);

System.debug('Seed data created');
```

C.4 Verification Queries and Anonymous Apex (Section 9)

SOQL used to check stored values, the latest location and status counts for reporting, followed by an Execute Anonymous snippet that calls the invocable method directly. Each statement is run separately.

```
// 1) Check the stored values for a parcel
SELECTId,Name,Parcel_ID__c,Status__c, Weight__c, Estimated_Delivery_Date__c, Sender__c
FROM Parcel__c
WHERE Parcel_ID__c = 'P-001'

// 2) Latest delivery milestone for a parcel
SELECTParcel__c,Current_Location__c,Estimated_Delivery__c, LastModifiedDate
FROM Delivery__c
WHERE Parcel__r.Parcel_ID__c = 'P-002'
ORDER BY LastModifiedDate DESC
LIMIT 1

//3)Countparcelsperstatus(basisfor a report or dashboard)
SELECT Status__c, COUNT(Id) total
FROM Parcel__c
GROUP BY Status__c

//4)ExecuteAnonymous:calltheinvocable method directly and print the result
ParcelTrackerController.ParcelStatusInput i = new ParcelTrackerController.ParcelStatusInput();
i.trackingNumber = 'P-001';
List<ParcelTrackerController.ParcelStatusOutput> res =
      ParcelTrackerController.getParcelDetails(
            newList<ParcelTrackerController.ParcelStatusInput>{ i });
System.debug(JSON.serializePretty(res));
```

C.5 Prompt Template: Retrieve Parcel Details (Section 5)

The template body as configured, followed by recommended wording that addresses defect DEF-02.

```
Templatename:Retrieve Parcel Details
APIname :Retrieve_Parcel_Details
Type :Flex / record-grounded prompt template (object: Parcel__c)
Input :Record Name (Parcel__c)

--- Prompt body ------------------------------------------
Parcel Tracking Update
- Parcel Name: {!$Input:Parcel__c.Name}
- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c}
-EstimatedDelivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}

---Recommendedhardening (add to the body) ----------------- Respondwithexactly the five
lines above, in the same order, usingonlythevalues supplied. Do not add greetings,
closing remarksoranyinformation that is not listed. If a value is
blank, write "Not available".
```

C.6 Agent Subagent Instructions: Parcel Tracker (Section 7)

```
Subagent          :ParcelTracker
Description       :Thiswilltracktheparcelbasedontheparcel id

--- Reasoning instructions (recommended) --------------------
1. When the user asks about a parcel, extract the Parcel ID
     (for example P-001). If no ID is given, ask the user for it.
2. Call the action "Get Parcel Tracking Details" with that ID.
     Never answer from general knowledge.
3. Report only the values returned by the action: parcel name,
                  status, weight, estimated delivery date and current location.
                4. If the action returns found = false, reply that no parcel was
     found with that ID and ask the user to check it.
5. If current location is blank, say that no location has been
     logged yet. Do not guess a location.
6. Do not reveal sender or receiver contact details.
```

C.7 Flow Specification: Parcel Details – V1 (Section 6)

Text specification of the flow, including the recommended Decision element for unknown IDs.

```
FLOW    : Parcel Details - V1                    Type: Autolaunched Flow (no trigger)
STATUS : Active (last saved 10/4/2026, 09:05 AM)

RESOURCES
   ids          Text          Available for input
   Output       Text          Available for output

ELEMENTS (in execution order)
   1. Autolaunched Flow (Start)
```

```
  2. Get Parcel Records          [Get Records]
        Object      :Parcel__c
        Filter      :Parcel_ID__c       Equals {!ids}
        Store       :Onlythefirstrecord -> variable "parcel"
  3.ParcelUpdates               [Action:prompt    template    Retrieve    Parcel    Details]
        APIname                 :Parcel_Updates
        objectToSummarize       :parcelfrom Get Parcel Records
        Outputlanguage          :notincluded
        PreviewOnly             :notincluded
  4. Assignment Output          [Assignment]
        API name : Assignment_Output
        Output      Equals      ParcelUpdates > Prompt Response
  5. End

RECOMMENDEDADDITION(TC-08,not-found handling)
  InsertaDecisionafter"GetParcel Records":
      Outcome"Found"            :parcelis not null -> continue to Parcel Updates
      Default"Notfound"         :Assignment Output = "No parcel found with that ID."
```

C.8 Permission Set Metadata: SwiftShip Permission Set (Section 8)

Metadata XMLgranting accessto the Apex class, theflow,read accesstoparcelanddelivery data, and only the tracking fields. The flow API name must be confirmed in Flow Builder.

```
<?xmlversion="1.0" encoding="UTF-8"?>
<PermissionSet xmlns="http://soap.sforce.com/2006/04/metadata">
    <label>SwiftShip Permission Set</label>
    <description>Run the parcel tracking flow and Apex action; read parcel data.</description>
    <hasActivationRequired>false</hasActivationRequired>

    <classAccesses>
        <apexClass>ParcelTrackerController</apexClass>
        <enabled>true</enabled>
    </classAccesses>

    <!--Verify the flow API name in Flow Builder > flow properties -->
    <flowAccesses>
        <enabled>true</enabled>
        <flow>Parcel_Details</flow>
    </flowAccesses>

    <objectPermissions>
        <allowCreate>false</allowCreate>
        <allowDelete>false</allowDelete>
        <allowEdit>false</allowEdit>
        <allowRead>true</allowRead>
        <modifyAllRecords>false</modifyAllRecords>
        <object>Parcel__c</object>
        <viewAllRecords>false</viewAllRecords>
    </objectPermissions>
    <objectPermissions>
        <allowCreate>false</allowCreate>
        <allowDelete>false</allowDelete>
        <allowEdit>false</allowEdit>
        <allowRead>true</allowRead>
        <modifyAllRecords>false</modifyAllRecords>
        <object>Delivery__c</object>
        <viewAllRecords>false</viewAllRecords>
    </objectPermissions>

    <!--Field-level security: tracking fields readable; contact fields NOT granted -->
    <fieldPermissions>
        <editable>false</editable>
        <field>Parcel__c.Parcel_ID__c</field>
        <readable>true</readable>
    </fieldPermissions>
    <fieldPermissions>
        <editable>false</editable>
        <field>Parcel__c.Status__c</field>
        <readable>true</readable>
    </fieldPermissions>
    <fieldPermissions>
        <editable>false</editable>
        <field>Parcel__c.Weight__c</field>
        <readable>true</readable>
    </fieldPermissions>
    <fieldPermissions>
        <editable>false</editable>
        <field>Parcel__c.Estimated_Delivery_Date__c</field>
        <readable>true</readable>
    </fieldPermissions>
    <fieldPermissions>
        <editable>false</editable>
        <field>Delivery__c.Current_Location__c</field>
```

```
        <readable>true</readable>
    </fieldPermissions>
    <fieldPermissions>
        <editable>false</editable>
        <field>Delivery__c.Estimated_Delivery__c</field>
        <readable>true</readable>
    </fieldPermissions>
</PermissionSet>
```

C.9 Deployment Manifest: package.xml (Section 14)

```
<?xmlversion="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
    <types>
        <members>ParcelTrackerController</members>
        <members>ParcelTrackerControllerTest</members>
        <name>ApexClass</name>
    </types>
    <types>
        <members>Parcel__c</members>
        <members>Delivery__c</members>
        <members>Sender__c</members>
        <members>Receiver__c</members>
        <name>CustomObject</name>
    </types>
    <types>
        <members>Parcel_Details</members>
        <name>Flow</name>
    </types>
    <types>
        <members>Retrieve_Parcel_Details</members>
        <name>GenAiPromptTemplate</name>
    </types>
    <types>
        <members>SwiftShip_Permission_Set</members>
        <name>PermissionSet</name>
    </types>
    <!--Set to the API version of your org / CLI -->
    <version>65.0</version>
</Package>
```

C.10 Sample Action Input and Output (Section 7)

Illustrativerequestand responses fortheApexaction.Thevaluesfor P-001 are those verified in the org; the response for P-999 shows the not-found case.

```
//Request sent by the agent action (illustrative)
{
  "requests": [ { "trackingNumber": "P-001" } ]
}

//Response when the parcel exists (values from test record P-001) [

  {
    "trackingNumber": "P-001",
    "found": true,
    "parcelName": "Laptop",
    "status": "Booked",
    "weight": "2",
    "estimatedDelivery": "9/17/2026",
    "currentLocation": null,
    "message": "Parcel found."
  }
]

//Response when the ID does not exist (TC-08)
[
  {
    "trackingNumber": "P-999",
    "found": false,
    "message": "No parcel was found with ID P-999."
  }
]
```

C.11 Initial Prototype Controller (Superseded)

The first versionof thecontroller, keptforversionhistory. It returns a fixed mock response for any non-blank tracking number, and its commented real-data option refers to fields that do not exist in the data model (Tracking_Number__c, Location__c, Activity__c). It was replaced by C.1.

```
public with sharing class ParcelTrackerController {
```

```
// Invocable Method - Agentforce Builder action-oda connect aagum entry point
@InvocableMethod(
    label='Get Activity Details'
    description='Fetchesstatus,currentlocation,andactivitydetailsforagivenparcel tracking
        number.'
)
publicstaticList<ParcelStatusOutput>getActivityDetails(List<ParcelStatusInput>requests) {
    List<ParcelStatusOutput> results = new List<ParcelStatusOutput>();

    for (ParcelStatusInput request : requests) {
        ParcelStatusOutput output = new ParcelStatusOutput();

        if (String.isNotBlank(request.trackingNumber)) {

            /*
             * OPTION 1: Salesforce Custom Object-la irundhu data fetch panna (Real Data):
             * Unskipping/Uncommenting this block if you have a custom object named Parcel__c:
             *
             *List<Parcel__c>parcels=[SELECTStatus__c,Location__c,Estimated_Delivery__c,
                  Activity__c
             *                                    FROMParcel__c
             *                                    WHERETracking_Number__c=:request.trackingNumber
             *                                    LIMIT1];
             * if (!parcels.isEmpty()) {
             *  *     output.trackingNumber=request.trackingNumber;
             *  *     output.status=parcels[0].Status__c;
             *        output.currentLocation=parcels[0].Location__c;
             *        output.estimatedDelivery=String.valueOf(parcels[0].Estimated_Delivery__c);
             }        output.activitySummary=parcels[0].Activity__c;
             */


            // OPTION 2: Mock response (Testing / Assessment setup-ku direct-a use pannalaam)
            output.trackingNumber = request.trackingNumber;
            output.status = 'In Transit';
            output.currentLocation = 'Regional Sorting Hub';
            output.estimatedDelivery = String.valueOf(System.today().addDays(2));
            output.activitySummary='Parcelscannedanddepartedfromfacilityforfinal delivery.';

        } else {
            output.status = 'Error';
            output.activitySummary = 'Tracking number was not provided or invalid.';
        }

        results.add(output);
    }

    return results;
}

// Agentforce Input Wrapper Class
public class ParcelStatusInput {
    @InvocableVariable(required=truedescription='ThetrackingIDornumberprovidedby the user')
    public String trackingNumber;
}

// Agentforce Output Wrapper Class
public class ParcelStatusOutput {
    @InvocableVariable(description='Tracking ID')
    public String trackingNumber;

    @InvocableVariable(description='Current delivery status')
    public String status;

    @InvocableVariable(description='Current physical location')
    public String currentLocation;

    @InvocableVariable(description='Expected delivery date')
    public String estimatedDelivery;

    @InvocableVariable(description='Detailed activity or status summary')
    public String activitySummary;
}
```

C.12 Initial Prototype Test (Superseded)

The firsttest class. It calls theold methodnamegetActivityDetails and asserts a field isSuccess that the class never defined, and it asserts the mock status value. It was replaced by C.2.

```
@isTest
private class ParcelTrackerControllerTest {
```

```
@isTest
staticvoid testGetActivityDetailsSuccess() {
    //Prepare Request
    ParcelTrackerController.ParcelStatusInput req = new ParcelTrackerController.ParcelStatusInput();
    req.trackingNumber = 'SWIFT-99281';

    List<ParcelTrackerController.ParcelStatusInput> reqList = new
        List<ParcelTrackerController.ParcelStatusInput>{ req };

    Test.startTest();
    List<ParcelTrackerController.ParcelStatusOutput> resList =
        ParcelTrackerController.getActivityDetails(reqList);
    Test.stopTest();

    System.assertEquals(1, resList.size());
    System.assertEquals(true, resList[0].isSuccess);
    System.assertEquals('In Transit', resList[0].status);
}

@isTest
staticvoid testGetActivityDetailsInvalidInput() {
    ParcelTrackerController.ParcelStatusInput req = new ParcelTrackerController.ParcelStatusInput();
    req.trackingNumber = '';

    List<ParcelTrackerController.ParcelStatusInput> reqList = new
        List<ParcelTrackerController.ParcelStatusInput>{ req };

    Test.startTest();
    List<ParcelTrackerController.ParcelStatusOutput> resList =
        ParcelTrackerController.getActivityDetails(reqList);
    Test.stopTest();

    System.assertEquals(false, resList[0].isSuccess);
    System.assertEquals('Error', resList[0].status);
}
```

Appendix D. Glossary

Term	Meaning
Agentforce	Salesforce platform for building AI agents that take actions in response to natural language.
Agent Builder	Tool for configuring agents, subagents (topics), instructions and actions.
Apex	Salesforce server-side programming language.
Invocable method	Apex method that flows and agents can call as an action.
Auto-Number	Field type that assigns an incrementing identifier such as P-001.
FLS	Field-Level Security: controls which profiles or permission sets can see or edit a field.
Flow Builder	Low-code tool for automating business processes.
Grounding	Supplying an AI model with real record data so that answers reflect stored facts.
Hallucination	A confident AI answer that is not supported by data.
LLM	Large language model.
Lookup	Relationship field that references a record of another object.
Merge field	Placeholder in a template replaced with a record value at run time.
Permission set	Collection of permissions added to users on top of their profile.
Prompt Builder	Tool for creating reusable, grounded prompt templates.
SOQL	Salesforce Object Query Language.
WITH USER_MODE	SOQL clause that enforces the running user's object and field permissions.
Subagent / topic	Group of instructions and actions within an agent for one kind of task.

Table 35.Glossary